Warsaw University of Technology

Faculty of Mathematics
and Information Science

Bachelor's diploma thesis

in the field of study Computer Science and Information Systems

Design and Implementation of a Collaborative Board Using the Local-First Approach

Piotr Jankiewicz

student record book number 288767

thesis supervisor

dr inż. Paweł Kotowski

WARSAW 2026

Abstract

Design and Implementation of a Collaborative Board Using the Local-First Approach

This thesis concerns the design and implementation of a collaborative whiteboard application built around the local-first approach. The solution relies on distributed conflict-free replicated data types (CRDTs) and real-time peer communication over WebRTC to make collaboration with other users feel seamless, even in the face of intermittent connectivity.

The end result is a desktop application that lets multiple users draw and erase on a shared whiteboard at the same time. Beyond describing the concrete implementation, this paper also touches on the broader topic of local-first design philosophies and why they are increasingly relevant for modern collaborative software.

Key words: rust language, local-first, CRDT, WebRTC, distributed system, star topology, multithread real time communication

Streszczenie

Projekt i wdrożenie kolaboratywnej tablicy z wykorzystaniem podejścia „local-first"

Celem pracy inżynierskiej jest zaprojektowanie i implementacja kolaboratywnej tablicy z wykorzystaniem podejścia "local-first". Wypracowane rozwiązanie używa rozproszonego bezkonfliktowego typy danych (CRDTs) oraz komunikacji w czasie rzeczywistym przez protokół WebRTC, by umożliwić bezproblemową kolaborację z innymi użytkownikami. Ostatecznym wynikiem jest aplikacja desktop-owa umożliwiająca rysowanie oraz zmazywanie poprzednich pociągnięć na białej tablicy z wieloma użytkownikami jednocześnie. Praca pokrywa opis rozwiązania ale również porusza bardziej obszerny temat architektur typu local-first w aplikacjach kolaboratywnych.
Słowa kluczowe: język rust, local-first, CRDT, WebRTC, system rozproszony, topologia gwiazdy, wielowątkowa komunikacja w czasie rzeczywistym

Contents

1. Introduction

1.1. Analysis and description of collaborative local-first applications

Collaborative applications have not always been so popular as they are these days. You might not even know that some of applications that we use every day are collaborative. In everyday life of a software developer almost all of modern IDEs have collaborative work options, almost all project management software are also collaborative. These are examples close to software engineers, but these applications do not limit themselves to software development. Google Docs, Microsoft 365, Figma, Miro and many more are examples of collaborative applications that are used by millions of people every day. These applications are not only used for work, but also for education and entertainment.

It hasn't always been like this. The steep rise of collaborative applications began as the remote work became more popular but also it owes its rise of popularity to universality of fast global network and increase of computing power of everyday devices.

And the apps for collaborative work today couldn't exist 20 years ago. They haven't been simply invented yet. When online applications first came out, their architectures were really simple. In fact, online applications were nothing more than client-server applications. On the Server side, data was stored and processed; On the Client side, the data was presented. The library is used to send data to the user and to send user input to the server. This architecture had its advantages and disadvantages. It was simple and easy to implement, but it had a single point of failure. When the server went down, the entire web site went down. It also wasn't very scalable because all requests came from the clients to the one server.

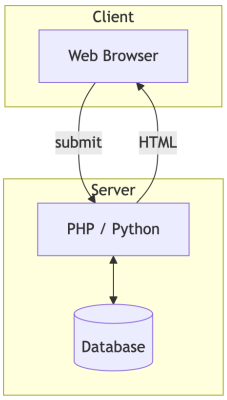

Figure 1.1: Simple client-server architecture of early internet era applications

These kind of application belong to "cloud-first" family. There was no need for "local-first" architectures yet and this term has not been popularized. Before even the discussion on topic of

"local-first architectures" the "cloud-first" approach had its own more than one decade of fast development. From simple client server architectures the system rose to more difficult forms of large size client-side application with lots of layers and microservices. Picture below shows architectures that emerged on the market with rise of popularity of mobile and web apps from 2010.

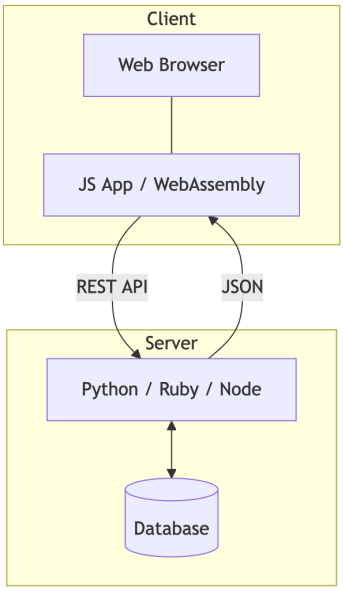

Figure 1.2: More complicated architecture of modern mobile/web applications

How far are these architectures from beeing good fit for collaboarative appliactions? Lets have a look on typical modules that take part in standrd user interface interaction. Picure below shows typical modules of morden web/mobile/desktop cloud-application.

Figure 1.3: Typical modules of modern cloud-based applications

As we can see, the level of complexity is already very high. A single user interaction triggers a cascade through multiple functions, modules, and services. Trying to layer real-time collaboration on top of this kind of architecture is, frankly, a nightmare - it would require coordinating asynchronous, time-consuming remote calls across all those services just to keep everyone in sync.

For a while people did try exactly this. The approach was called OT - operational transformation - and it is still the backbone of Google Docs today. OT works, but it is notoriously hard to implement correctly, and the failure modes under concurrent edits are subtle. In this thesis I chose to focus on local-first architectures instead, which have gained much broader traction in the industry and open up a wider range of application types.

There are also other reasons to be skeptical of cloud-first when it comes to collaborative applications. Consider what actually happens during a typical user interaction. Below we can see the general asynchronous flow of a user interacting with a standard cloud-based application.

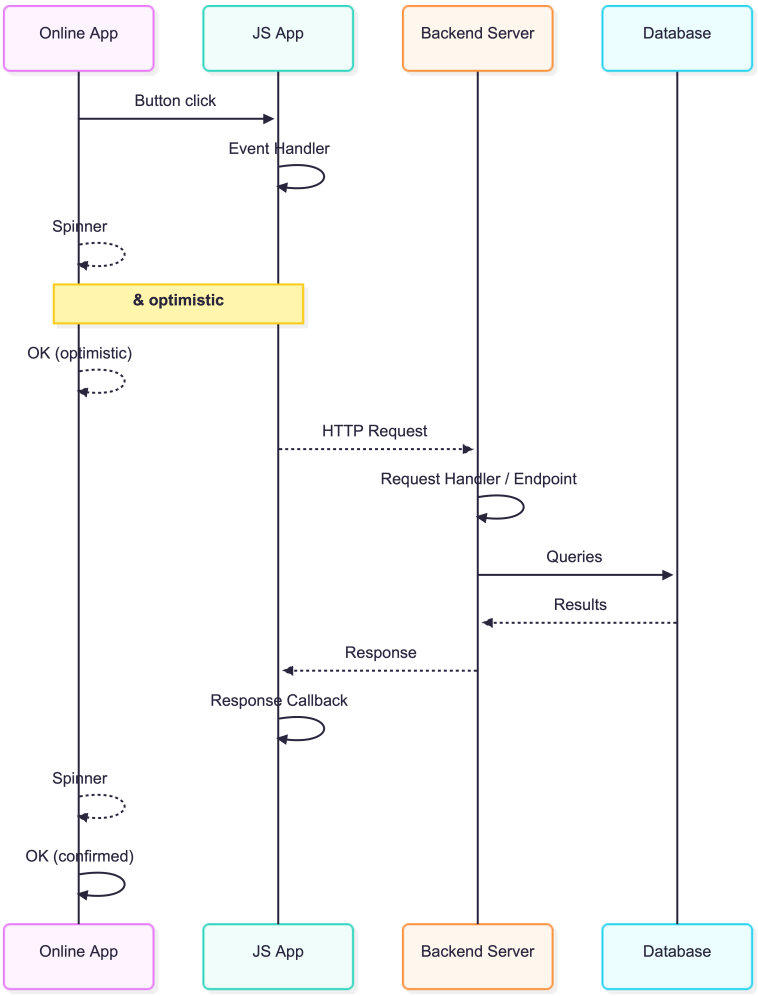

Figure 1.4: Typical sequance of interaction with a user in a modern cloud-based applications

What stands out immediately is how much the system depends on database connectivity. The obvious follow-up question is: what happens when there is no internet? The answer is simple - nothing works. In that sense the entire application is really just an asynchronous graphical wrapper around a remote database, which is a somewhat uncomfortable truth.

This brings us to a useful contrast. In the cloud-first model the underlying assumption is: "if the user's action wasn't saved to the database, it never happened." Local-first flips this on its head: "local persistence is what matters most - the cloud is there only to connect peers" [4, 3].

With that distinction in mind, let's look at how a typical user interaction plays out in a local-first system instead.

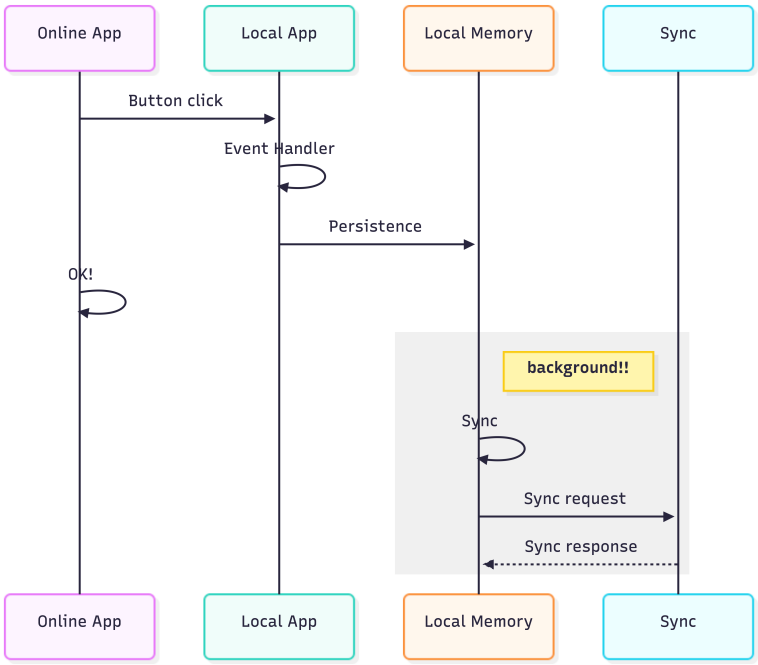

Figure 1.5: Typical sequance of interaction with a user in a local-first applications

The contrast is immediately visible. In a local-first system, the user is never blocked waiting for a database roundtrip - all interactions happen against the local replica, and connectivity to other peers is a bonus rather than a prerequisite. Synchronization of state between users happens on each machine independently. That is precisely what makes the system distributed in any meaningful sense. It is also worth noting that the architecture is considerably flatter: far fewer services and remote calls are involved. Later chapters will show concretely how this property is what makes true real-time collaboration actually achievable.

1.2. Local-first vs Cloud-first

Lets have a look on the main differences between local-first and cloud-first approaches [4, 3]. The table below shows the main differences between these two approaches when it comes to designing collaborative application.

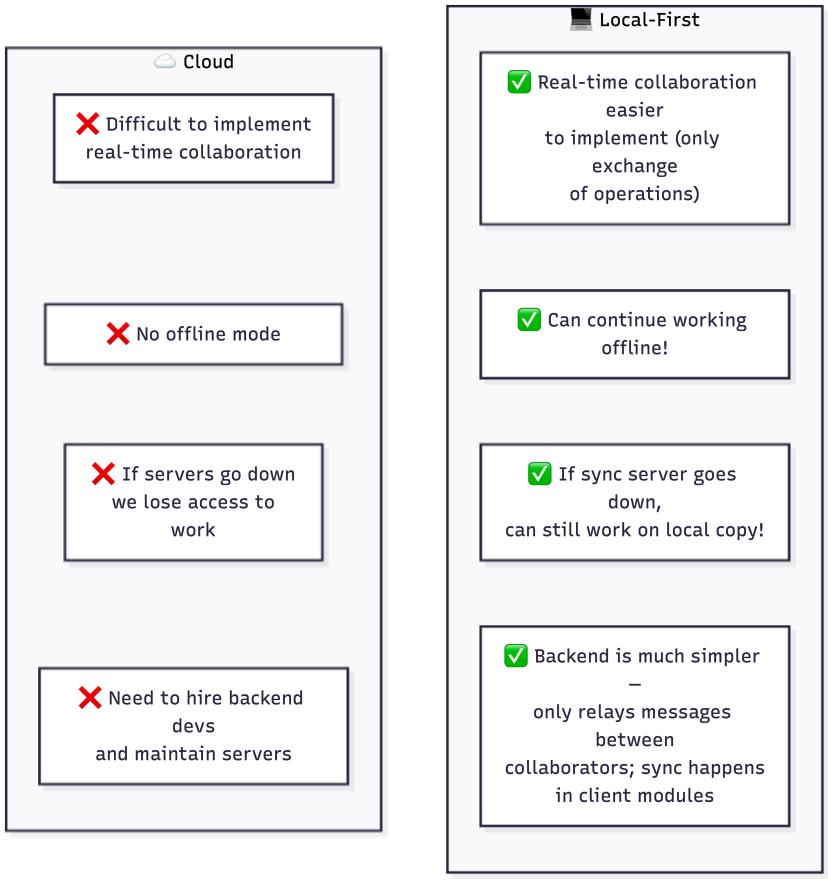

Figure 1.6: Cloud vs Local comparison

Given the trade-offs, local-first is the more compelling choice when designing a collaborative application. That said, it is not a universal solution - there are scenarios where it simply does not fit. The table below lays out use cases where local-first works well and where it breaks down.

Table 1.1: Use cases: Suitable and Unsuitable for Local-First Approach

Suitable for	**Unsuitable for**
Real-time collaborative editing (documents, whiteboards)	Applications requiring strong centralized control
Offline-first mobile/desktop apps	Systems with strict regulatory or audit requirements
Peer-to-peer file sharing	Applications with massive global data aggregation
Personal productivity tools with sync	Centralized transactional systems (e.g., banking)
Small/medium team collaboration	Apps needing immediate global consistency

1.3. Why would one implement own collaborative system ?

Building a collaborative whiteboard is in many ways just a gateway into a much richer topic: distributed, conflict-free, real-time data storage where every operation is reversible and mergeable. The whiteboard is an ideal teaching example because it is visually intuitive - you can see convergence happening in front of you - yet it exercises exactly the same data-structure challenges you would face in far more complex systems.

The same underlying principles - distributed replicas that stay consistent without a central arbiter - show up in wildly different domains. Drone swarms that broadcast formation data to one another, peer-to-peer file-sharing networks, collaborative code editors, even multi-player games: all of them are, at some level, the same problem with a different domain on top. The motivation for building such systems oneself is articulated well by the seven ideals of local-first software [3]: fast local interaction, multi-device access, offline operation, seamless collaboration, longevity of data, privacy by default, and ultimate user ownership - properties that off-the-shelf cloud SaaS rarely delivers in full.

1.4. Examples from industry

Known examples of local-first applications are: Figma, Miro, Notion, Obsidian, Roam Research, Coda, Google Docs (although it is not pure local-first application). These applications are used by millions of people every day. They are used for work, education and entertainment.

1.5. Known architectures for collaborative applications

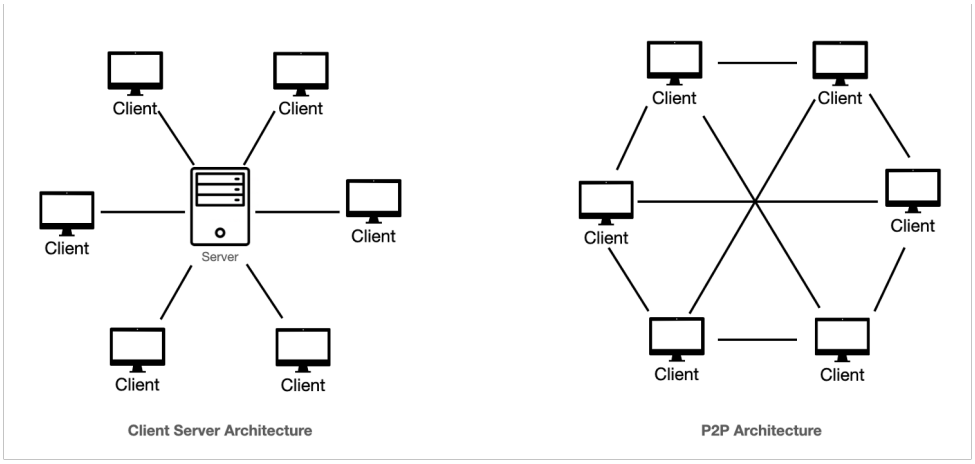

Figure 1.7: Comparison of Client-Server and Peer-to-Peer (P2P) Architectures

Connectivity among multiple users always comes down to one of two choices: peer-to-peer, where clients talk directly to each other [2], or a more centralised relay model such as star topology. Neither is strictly better - each makes different trade-offs. The table below is a straightforward comparison of the two when the goal is a local-first collaborative whiteboard.

Table 1.2: Comparison of Peer-to-Peer and Star Topology Architectures

Peer-to-Peer (P2P)	**Star Topology**
No need to create a server for communication. Communication is direct - good. Clients manage their connections themselves - might be problematic.	Central node relays communication. It requires support of server. Keeping servers running is expansive.
Communication might be faster because it is direct. Users might be close to each other.	Clients sometimes might unnesesarry communicate via very distant server to both of them, which would cause delays.
High resilience to single node failure	Central node is a single point of failure
Complex synchronization and conflict resolution - corrdination of connections has to be implemented on client side.	Easier management and coordination - server keeps all informations on connectivity. Clients connect only to one server.
Scalability can be challenging - many users might have very low maximum of connections it could sustain.	Scalability depends on central node capacity, but server can easily be scaled hirizontlay or verticaly
Suitable for decentralized systems with limited numer of peers at the same time (for example 10)	Suitable for medium (up to 50) teams with central coordination - for example video conferance rooms

As we can see - each solution suits different applications.

1.6. OT vs CRDTs

One of the more fundamental questions in building a collaborative system is how to handle conflicting edits. When two users modify the same piece of data at the same time, the system needs a principled way to decide what the merged result looks like. For a data structure to support this safely, every insert and delete operation must be **commutative**, **associative**, and **idempotent**. Meet those three conditions and eventual consistency follows naturally, assuming messages eventually get delivered.

Two families of algorithms address this. The older one is OT - operational transformation.

The idea is to transform operations relative to each other so that applying them in any order yields the same outcome. Google Docs is the most famous example and OT still powers it today. The trouble is that getting OT right is genuinely difficult - the edge cases multiply quickly, and bugs in the transformation logic can silently corrupt documents [2].

CRDTs took a different path. Instead of transforming operations at merge time, you bake the convergence rules directly into the data structure itself [3]. If you design the structure correctly, merging two independently-evolved replicas is always well-defined and always produces the same result, with no coordination needed. The trade-off is that CRDTs can be less expressive than OT in some scenarios, but for most collaborative editing use cases the fit is excellent. Figma, Miro, Notion, Obsidian, and Coda all rely on CRDTs - a reasonable signal that the approach has matured enough for production use. Recent research has further narrowed the gap between the two approaches: Eg-walker [5] achieves CRDT-like decentralisation with memory and loading performance competitive with centralised OT systems.

1.7. Vision of the System

The system I set out to build is a desktop application for real-time collaborative drawing on a shared whiteboard. Rust was chosen as the implementation language, primarily for its memory-safety guarantees and predictable performance. The user interface is built on top of egui, which turned out to be a good fit for a native desktop application that needs to stay responsive while background threads are busy with network I/O.

Peers communicate over WebRTC, with an open-source LiveKit server acting as the relay. Rather than requiring accounts or any kind of login, users simply type a room name to join a session - the room is created automatically if it does not exist yet. Whiteboard state is kept consistent across all participants via CRDTs from the Automerge library. In practice this means everyone can draw and erase at the same time without worrying about conflicts, and the cursors of remote users are shown live on the canvas.

Each stroke is represented as a vector of sampled points with an associated color and width - not as raw pixels. One deliberate design choice worth noting upfront: the application does not persist anything by default. When the app is closed, the session is gone unless the user explicitly exported it first. A mobile version was never part of the scope.

1.8. Requirements Specification

1.8.1. Functional Requirements

Below you can see tables with user stories for each actor in the system: user, application, and server.

Table 1.3: Functional Requirements - User Stories (Actor: User) Part 1

ID	As a...	I want to...	In order to...
US-01	User	launch the desktop app and see a blank whiteboard	start drawing immediately without any setup
US-02	User	draw freehand strokes on the whiteboard using a pen tool	express my ideas visually
US-03	User	erase previously drawn strokes	correct mistakes or modify drawings
US-04	User	change the color of the pen	differentiate between elements and add visual clarity
US-05	User	change the thickness (width) of the pen	draw both fine details and bold strokes
US-06	User	clear the entire whiteboard	start drawing from an empty board again
US-07	User	save the current whiteboard state to a file (.crdt/.png)	persist my work for later use
US-08	User	load a previously saved whiteboard document (.crdt/.png)	continue working on a previous drawing
US-09	User	create a new empty document	begin a fresh collaborative session or personal workspace
US-10	User	enter a room name and connect to a collaborative session	work together with other users in real time
US-11	User	disconnect from a collaborative session at any time	leave the room without closing the application
US-12	User	see the cursors of other connected users on the whiteboard	know where collaborators are drawing and coordinate visually
US-13	User	see strokes drawn by other users appear in real time	have a seamless collaborative experience
US-14	User	send chat messages to other participants in the session	communicate textually alongside visual collaboration
US-15	User	be notified when another participant joins or leaves the room	stay aware of who is currently collaborating
US-16	User	join a session without creating an account (only room code needed)	minimize friction and start collaborating instantly

Table 1.4: Functional Requirements - User Stories (Actor: Application, Part 1)

ID	As a...	I want to...	In order to...
AP-01	Application	maintain a local CRDT document (Automerge) representing the whiteboard state	enable offline-capable and conflict-free editing
AP-02	Application	render all strokes stored in the CRDT as raster graphics on a canvas	display the current whiteboard state to the user
AP-03	Application	generate CRDT sync messages when local changes occur	propagate the user's drawing operations to all connected peers
AP-04	Application	receive and apply incoming CRDT sync messages from remote peers	update the local whiteboard with changes made by others
AP-05	Application	establish a WebRTC data channel connection through the LiveKit server	enable real-time, low-latency peer-to-peer communication
AP-06	Application	broadcast the local user's cursor position to all connected peers	allow other users to see where the local user is pointing
AP-07	Application	receive and render remote cursor positions on the whiteboard	display collaborators' cursors with distinguishing color and name
AP-08	Application	run network I/O and CRDT synchronization on separate threads (Tokio)	keep the UI responsive and non-blocking during communication
AP-09	Application	serialize the CRDT document to a byte vector for file export	allow the user to save the full document state
AP-10	Application	deserialize a byte vector from a file into a CRDT document	restore a previously saved whiteboard with full edit history
AP-11	Application	read LiveKit connection credentials from an .env file or environment variables	configure server connectivity without hardcoding secrets

Table 1.5: Functional Requirements - User Stories (Actor: Application, Part 2, Actor: Server)

ID	**As a...**	**I want to...**	**In order to...**
AP-12	Application	resolve conflicts between concurrent edits automatically via CRDT merge	guarantee eventual consistency without manual conflict resolution
AP-13	Application	represent each stroke as a vector of points with associated width and color in the CRDT	faithfully reproduce strokes across all replicas
AP-14	Application	provide a modern responsive GUI via the egui library	deliver a native desktop experience on macOS and Windows
ID	**As a...**	**I want to...**	**In order to...**
SV-01	Server	manage rooms (channels) that users can join by name	group collaborating users into logical sessions
SV-02	Server	relay WebRTC data channel messages between all participants in a room (star topology)	ensure every client receives every peer's sync messages
SV-03	Server	authenticate incoming client connections using API key/secret token validation	prevent unauthorized access to collaboration rooms
SV-04	Server	notify all participants when a new user joins a room	allow clients to initiate CRDT sync with the newcomer
SV-05	Server	notify all participants when a user disconnects from a room	allow clients to remove the disconnected user's cursor and update UI
SV-06	Server	automatically create a room when the first participant connects to a new room name	eliminate the need for manual room provisioning
SV-07	Server	support multiple concurrent rooms with independent participant lists	allow several collaborative sessions to run simultaneously
SV-08	Server	forward data channel packets with minimal latency	support the real-time requirements of collaborative whiteboard synchronization

1.8.2. Non-Functional Requirements

Non-functional requirements define the quality attributes of the system. The FURPS+ model is used to categorize them into Functionality, Usability, Reliability, Performance, and Supportability. The table below presents the non-functional requirements of the collaborative whiteboard application.

Table 1.6: Non-Functional Requirements - FURPS+ Analysis (Part 1)

Category	ID	Description
Usability	NF-01	The user interface shall be simple and intuitive, allowing basic operations (drawing, erasing, connecting to a session) without the need for training.
	NF-02	The system shall clearly display the synchronization status (online/offline) and inform the user about connection state changes.
	NF-03	The editor shall provide visual indication of collaborators' cursors in real time.
	NF-04	The application interface shall be in English.
Reliability	NF-05	The system shall ensure continuity of work in offline mode through local CRDT state, with automatic synchronization upon reconnecting to the session.
	NF-06	The CRDT-based data model shall guarantee that no data is lost during concurrent edits - all user operations shall be preserved after merge.
Performance	NF-07	The synchronization time of changes between users shall not exceed 2 seconds on a connection with bandwidth $\geq$10 Mb/s.
	NF-08	Loading a document from local storage (.crdt file) shall not exceed 1 second.
	NF-09	The application shall support simultaneous editing by at least 3 users on one document without noticeable delays in the interface.
	NF-10	The drawing canvas shall maintain a responsive frame rate ($\geq$30 FPS) during normal drawing operations.

Table 1.7: Non-Functional Requirements - FURPS+ Analysis (Part 2)

Category	ID	Description
Supportability	NF-11	The system documentation shall include installation and configuration instructions (README) to enable returning to the project after an extended period.
	NF-12	The application shall be buildable and runnable on macOS and windows.
Constraints (+)	NF-13	The application shall be implemented in Rust, using egui for the GUI, Automerge for CRDTs, Tokio for async runtime, and LiveKit for WebRTC signaling.
	NF-14	The application shall not persist any data by default - all state is lost when the app is closed unless explicitly exported by the user.

1.9. Business Cases

Collaboration is neccecary in any business. Companies are build on collaboration and division of work. It is a common effort to build something greater togheter. Therefore having tools that brings real time collaboration is neccecary. The most exiting tools that I think will emarge on the market in coming years are collaborative CAD/CAM applications for engineers. CRDTs might have its best years ahead of them.

1.10. User Interface vision

The assumptions for the user interface are as follows: whiteboard canvas takes up the majority of the screen, with a toolbar in the top bar containing pen and eraser tools, color and thickness selectors, and buttons for creating new documents, saving/loading and menu botton.

The menu side panel on the left can be toggled to show connectivity options, including a text field for entering a room/opening connection panel with information on other users and chat. Cursors of other users are shown as colored circles with their ids next to them at the bottom bar. The design is clean and minimalistic, focusing on ease of use and real-time collaboration.

Although the final version of the user interface differs from the initial vision, the main assumptions were met. The first design is shown in the picture below.

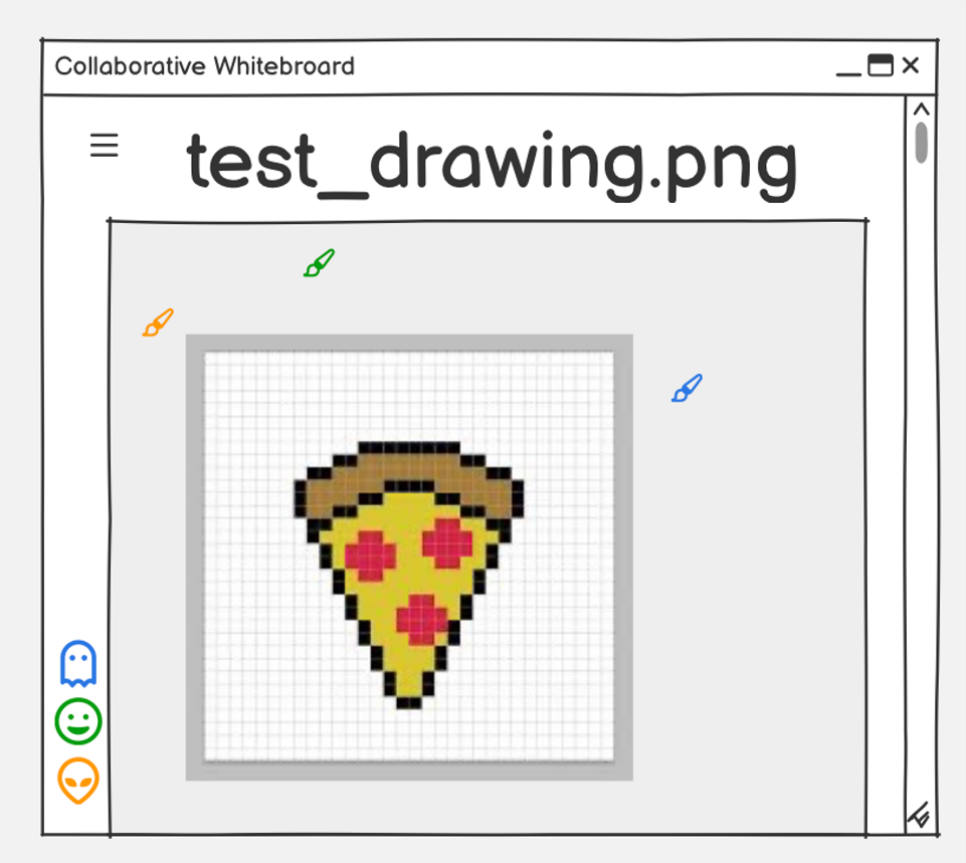

Figure 1.8: Initial vision of the user interface

2. Description of the solution

2.1. System Architecture

As mentioned in the introduction, the system is designed with a star topology architecture, where a central server (open source LiveKit SFU) facilitates communication between clients. Each client maintains a local CRDT document representing the whiteboard state, and synchronization occurs through WebRTC data channels. The architecture is modular, with clear separation of concerns between the user interface (egui), the CRDT data model (Automerge), and the networking layer (LiveKit/Tokio).

The program is devided mostly into three main modules: `ui.rs` which holds most of the application interface state and UX interaction methods, `backend_api.rs` - a module of abstraction which defines connection points of canvas/CRDT and networking, because what user sees on the screen is not just a matrix of pixels from persistent storage, but a result of rendering of the CRDT document.

`automerge_backend.rs` - a module which defines the CRDT document and methods of interaction with it. It is an implementation of `backend_api.rs` with a little bit of extra methods.

2.2. Technology Stack

The stack I decided to implement the solution in is mostly based on my intuition of current trends on software engineering market, therefore I decided to use RUST language - a modern and unusual language, which has gained some popularity in recent years, mostly due to its great implementation of memory safety / ownership of variables. Rust is a language which has a promise of solving approximately 60-70% of vulnerabilities in C/C++ systems, which are memory safety vulnerabilities.

Becasue the application is a desktop application, I decided to use egui library for the user interface. It is a modern and responsive GUI library for Rust, which allows to create native desktop applications with ease. It is also cross-platform, which means that the application can be run on both macOS and Windows without any changes. Possibly it could also be compiled to webassembly and run in the browser, but it is not a goal of this project.

When deciding on how to implement / use CRDTs in the project, the first idea was to use own implementation of CRDTs, more specifically a RGA - Replicated Growable Array and keep strokes as pixels in a matrix (array of arrays), but during the research, I stumbled upon a very highly regarded and industry wide accepted library developed by Cambride Cumputer Scientist

- Martin Kleppmann. The library is Automerge and it implements most of basic data structures as CRDTS. In my case I would use mostly use only few of them, but the library still provides a lot of useful methods for merging, generating sync messages, applying sync messages and so on [1].

While doing research on how to implement the communication layer, I found out that there is a very good open source project called LiveKit, which is a WebRTC SFU (Selective Forwarding Unit) server. Moreover - the LiveKit provides a Rust SDK. It provides a simple API for creating rooms and managing connections between clients. It also supports data channels, which are perfect for our use case of synchronizing CRDT documents. Therefore, I decided to use LiveKit as the communication layer for the application - both on client side with usage of Rust livekit SDK and on server side with LiveKit SFU as a docker container, which might be easly scaled if needed horizontaly with usage of Kubernetes and load balancers [6].

The communication needs to be efficient and non-blocking, therefore I decided to use Tokio - a Rust library for asynchronous programming. It provides a runtime for executing asynchronous tasks and allows for efficient handling of I/O operations, which is crucial for real-time communication in a collaborative application.

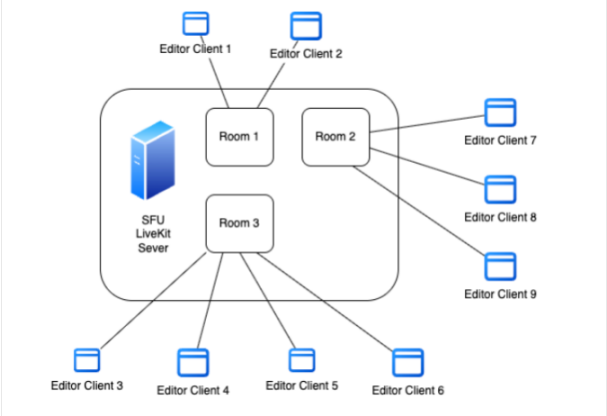

Figure 2.1: Architecture of the envisioned system

2.3. Application Modules

2.3.1. User Interface Modules and Implementation

Most of application state is kept in `AppView` struct. As we can see on the diagram below, it is a struct which holds all information about the state of the application: backend state (`backend`), status bar state (`status`), sidebar state (`SidebatState`), current page state (`page`), whiteboard state (`whiteboard`), LiveKit connection state (`livekit_`), remote cursors state (`remote_cursors`), and so on. It also holds methods for interaction with the user interface,

for example methods for rendering the UI (enforced by `egui`), handling user input, and so on. The main method of this struct is `update`, which is called in a loop and is responsible for rendering the UI and handling user input. The outline of the struct is shown in the diagram below. The main point of this struct is to be a single source of truth for the state of the application and to provide methods for interaction with the UI.

Figure 2.2: User Interface Modules 1

Below you can see more precise diagrams of submodules of the user interface module. `Page` enum defines the current page of the application, which can be either `Editor` (whiteboard) or `LiveKit` (connection panel). `SidebarState` struct defines the state of the sidebar, which can be either `visible` or not and have `default_width`. Another important enum is `Tool`, which defines the current tool selected by the user - either `Pen` or `Eraser`.

The `WhiteboardState` struct, shown in the diagram below, holds everything that describes the drawing canvas at any given moment. The two most important fields are `image` (a `ColorImage` pixel buffer representing the current raster state of the whiteboard) and `texture` (an `Option<TextureHandle>` that egui uses to upload the buffer to the GPU for rendering). On top of those, the struct keeps track of the user's current tool (`Pen` or `Eraser`), the selected color (`stroke_color`), line thickness (`stroke_width`), and an optional background image (`Option<ColorImage>`) that can be loaded from a file.

Perhaps the most interesting field is `current_stroke`: a `Vec<Point>` that accumulates the points of whatever stroke the user is currently drawing, before it gets committed to the CRDT document. This separation is what allows the drawing to feel immediate - the in-progress stroke is rendered locally on every frame, while the finalized version is pushed to Automerge and synchronized with peers only when the user lifts the pen.

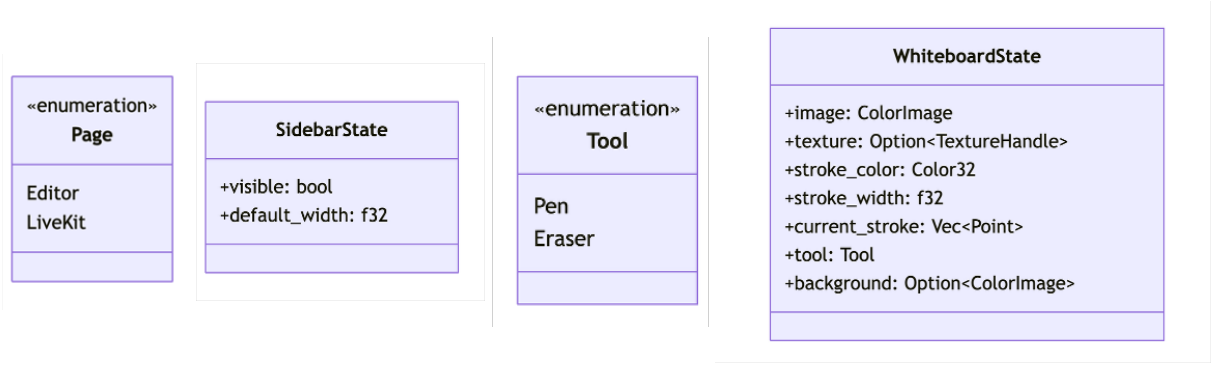

Figure 2.3: User Interface Modules

2.3.2. Backend Module

The backend module defines the boundary between the user interface and the CRDT synchronization logic. At its core lies the `DocBackend` trait (interface), which abstracts all document operations behind a unified API. This separation ensures that the UI layer does not depend on any specific CRDT implementation and can work with any backend that satisfies the trait contract.

The `DocBackend` interface exposes methods grouped into three categories:

- **Document manipulation** - The `apply_intent` method accepts an `Intent` (either `Draw` or `Clear`) and returns a `FrontendUpdate` containing the current list of strokes to render. The `get_strokes` method allows the UI to retrieve the full set of strokes at any time.
- **Synchronization** - The `peer_connected` and `peer_disconnected` methods notify the backend when remote peers join or leave. The `receive_sync_message` method processes incoming binary sync messages from a peer and returns a `FrontendUpdate` reflecting any changes. On the other hand, `generate_sync_message` produces an outgoing binary message broadcasted to all peers, returning `None` when there is nothing to send.
- **Persistence** - The `save` method serializes the entire document state into a byte vector, while `load` restores the document from previously saved bytes. The 1.0.0+1 version of the applicaiton in ALFA environment enables two file types persistence: .crdt and .png. These methods enable explicit file export and import without requiring any default persistence.

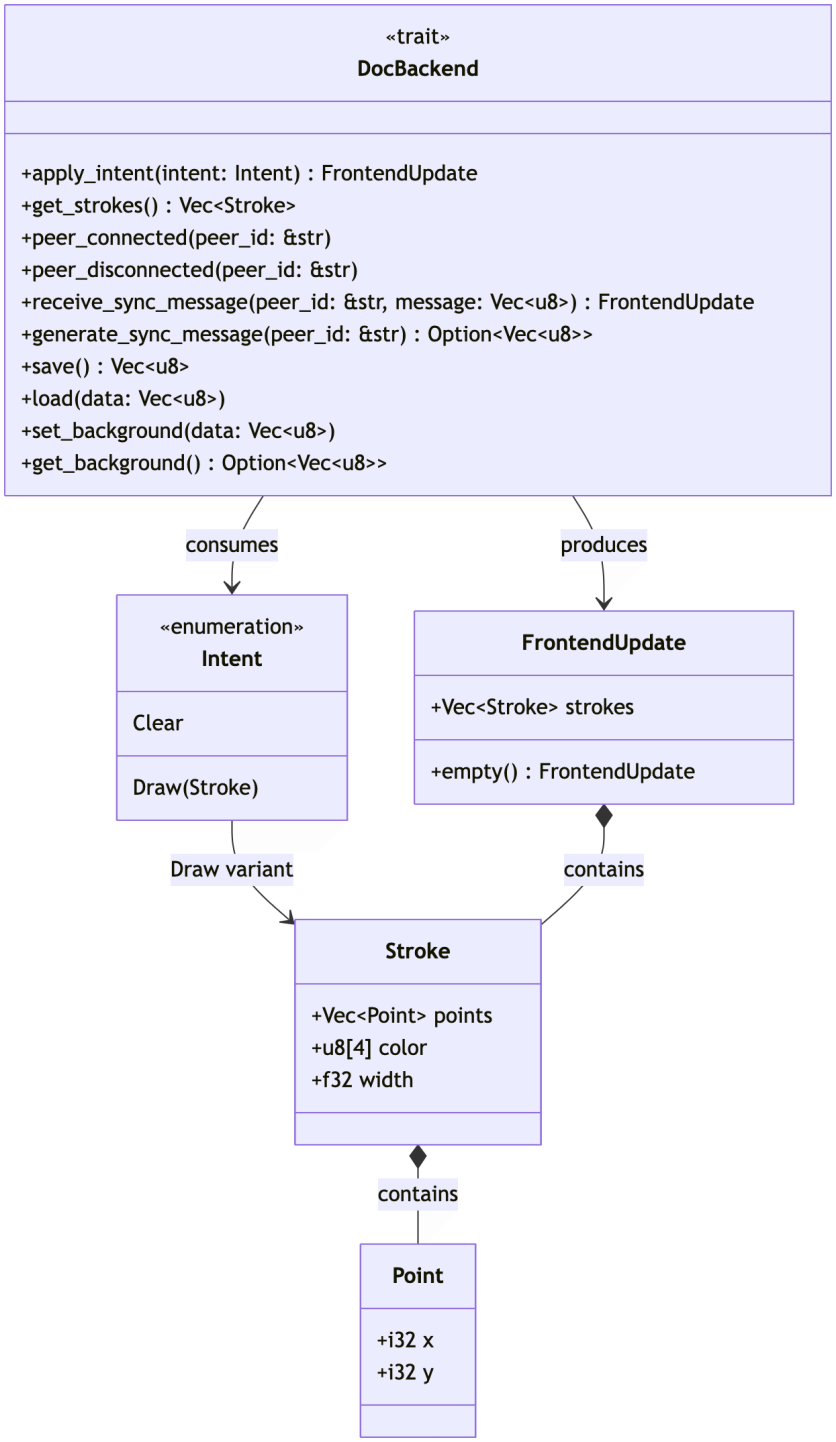

Figure 2.4: Backend Interface

The raster graphics data types are also defined in this module. `Point` represents a 2D coordinate, `Stroke` groups a sequence of points with a color (RGBA) and width, `Intent` encodes user actions, and `FrontendUpdate` carries the resulting list of strokes back to the UI.

This trait-based design makes the backend easily testable and replaceable - one could swap Automerge for a different CRDT library (or even a simple in-memory store for testing) without

modifying the user interface code.

2.4. Synchronization Protocol - How consistency is achieved

With `ui.rs` and `backend_api.rs` covered, we can now get into the actual meat of the system - the `automerge_backend.rs` module, which is where consistency is actually enforced.

I find it helpful to think about the synchronization architecture as three nested questions. Section 2.4 deals with the *how*: the Automerge sync protocol and the guarantees it provides. Section 2.5 answers *what* is being synchronized - the concrete document schema and data structures. Section 2.6 covers the *where*: the transport plumbing (LiveKit, WebRTC, Tokio) that actually carries the bytes between peers.

2.4.1. State-based vs Operation-based synchronization

Automerge is technically a state-based CRDT, but its sync protocol behaves much more like a delta-based system [1, 9]. Rather than broadcasting the full document on every change, it tracks exactly what each peer has already acknowledged and sends only the missing pieces.

Each peer keeps a `sync::State` per remote peer - a bookmark recording what that remote last saw. When a local change occurs, `generate_sync_message()` consults that bookmark, bundles only the unseen deltas into a compact binary blob, and returns it for transmission. On the receiving side, `receive_sync_message()` decodes the blob, merges the changes into the local document, and advances the bookmark so the same delta is never requested twice.

The diagram below shows this for a simple two-peer session. A draws something, packages the diff, sends it to B. B merges it and sends back its own pending changes if it has any. They keep exchanging diffs until both replicas match.

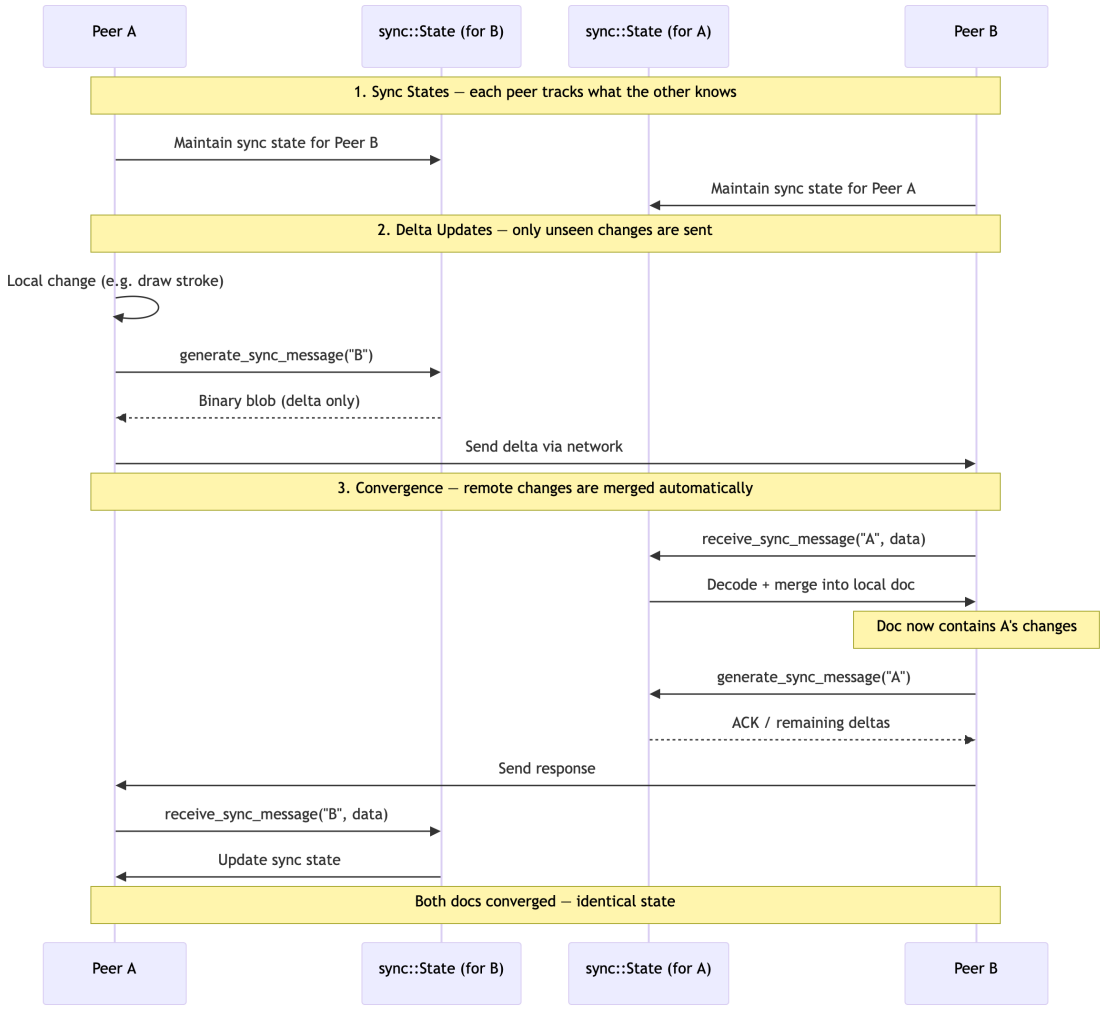

Figure 2.5: Automerge synchronization sequence between two peers

2.4.2. Automerge sync protocol

At the protocol level, synchronization in Automerge boils down to a conversation between two peers that keeps going until neither has anything new to share. The two functions that drive this are `generate_sync_message` and `receive_sync_message`.

1. **The core handshake**

   `generate_sync_message(peer_id)` asks the library: "what has changed since this peer last synced?" Automerge consults the per-peer `sync::State` and, if there are unseen deltas, packs them into a compact binary blob. If the peer is already current it returns `None` and the conversation ends. `receive_sync_message(peer_id, message)` does the other half: unpacks the blob, merges the changes into the local document, and advances the `sync::State` so those same changes are not re-sent next time.

2. **Bloom filters - cheap history summaries**

Sending a full history of every operation would be expensive, especially for peers that have diverged over a longer period. Automerge avoids this by including a Bloom filter in each sync message - a compact probabilistic structure that summarises all the operations the sender holds. The receiver compares its own operations against this filter and can quickly identify which of its changes the other side has never seen, bundling only those into the response.

3. **Why convergence sometimes takes several rounds**

   A single exchange rarely covers everything when peers have diverged significantly. The typical sequence looks like this:

   - **Round 1 (Discovery):** A sends its current heads and a Bloom filter.
   - **Round 2 (Response):** B identifies the gaps and sends the missing changes plus its own filter.
   - **Round 3 (Convergence):** A applies B's changes and sends whatever B was still missing.
   - **Round 4 (Finalization):** B applies those, sends an empty sync message as acknowledgement. Both sides are now identical.

The following Rust snippet shows how this iterative negotiation looks in code:

```rust
let mut peer1_state = sync::State::new();
let mut peer2_state = sync::State::new();

loop {
    // 1. Peer 1 generates a message for Peer 2
    let msg1 = doc1.sync().generate_sync_message(&mut peer1_state);

    if let Some(m) = msg1 {
        // Peer 2 applies changes from Peer 1
        doc2.sync().receive_sync_message(&mut peer2_state, m)?;
    }

    // 2. Peer 2 generates a message for Peer 1
    let msg2 = doc2.sync().generate_sync_message(&mut peer2_state);

    if let Some(m) = msg2 {
        // Peer 1 applies changes from Peer 2
```

```
            doc1.sync().receive_sync_message(&mut peer1_state, m)?;
        }

        // Stop when both sides have nothing left to send (convergence
            reached)
        if msg1.is_none() && msg2.is_none() {
            break;
        }
    }
```

This loop ensures that even if one message doesn't carry all necessary changes (due to size limits or dependency resolution), subsequent messages will complete the synchronization.

2.4.3. Conflict resolution and guarantees

In Automerge (and most CRDTs), conflict resolution is governed by the principle of **strong eventual consistency** [9]. This means that if two peers perform different actions on the same piece of data at the same time, they must both arrive at the exact same result without a central server deciding the outcome. Automerge achieves this through deterministic rules embedded directly into the data structure operations.

The "Clear" operation (Delete vs. Edit)

In Automerge, a delete operation (such as clearing a list or removing an element) specifically targets the set of *visible Change IDs* that the peer has observed at the moment the operation is issued.

- **Scenario**: Peer A clears the canvas (deletes all strokes from the list). Simultaneously, Peer B draws a new stroke.
- **Logic**: Peer A's "Clear" command only knows about the strokes that existed at the moment Peer A clicked clear. It has no knowledge of the new stroke Peer B is currently creating, because that stroke has not yet been synchronized.
- **Result**: The new stroke from Peer B *survives*. The old strokes are deleted (since both A and B agree they existed), but the new stroke remains because Peer A's delete operation did not "reach" it. This is the **Add wins** semantics - insertions that a peer has not observed cannot be removed by that peer's concurrent delete.

The "Draw" operation (Concurrent edits to the same property)

If both peers modify the same property concurrently (for example, changing the color of the same stroke at the same time - hypothetically, because 1.0.0 version of application does not allow such operations), Automerge must deterministically choose which value to display.

- **Scenario**: Peer A sets a stroke color to Red. Peer B sets the same stroke color to Blue.
- **Logic**: Automerge performs a deterministic **Last Write Wins (LWW)** tie-break based on a lexicographical comparison of the internal Change IDs (hashes).
- **Result**: If Peer A's change hash is `0xabc...` and Peer B's is `0xdef...`, Peer B wins because its hash is lexicographically higher. Crucially, both peers will see the same winner. No data is actually lost in the operation history - Automerge preserves the full history - but only one value is displayed as the "current" state.

The following table summarizes the conflict resolution strategies:

Table 2.1: Conflict resolution strategies in Automerge

Action	**Conflict Type**	**Resolution**
Clear	Add vs. Remove	Add wins. Deletes only affect elements the peer observed at the time of deletion.
Draw	Value vs. Value	Deterministic tie-break. The change with the higher lexicographical hash becomes the visible value.

Implications for the whiteboard application

In the Rust implementation, strokes live in an Automerge `List`. Both `splice` (for clearing) and `insert` (for drawing) are safe to call concurrently, and the semantics work out intuitively: if one user clears the whiteboard while another is mid-stroke, that new stroke survives on both screens once sync completes, while the old strokes are gone. This is a good outcome - a "Clear" can never silently destroy work that the clearing peer has not even seen yet.

If you wanted "Clear" to mean something stronger - "wipe absolutely everything, even changes I have not received yet" - that would need application-level logic, such as a `last_cleared_timestamp` used to filter out older strokes. For the current version I chose to stay with the simpler CRDT-native add-wins semantics, which is the safer and more predictable default for a collaborative drawing canvas.

2.4.4. Initial synchronization of a new peer

One thing I was genuinely curious about before implementing this was how a late-joining peer bootstraps itself. It has no local state and potentially needs to catch up to a session that has been running for a while.

It turns out the Automerge protocol handles this without any special-case logic at all. When a new participant appears, the existing peers detect the `ParticipantConnected` event from LiveKit and kick off the standard sync handshake. Because the newcomer starts with an empty Bloom filter, the very first sync message from any existing peer carries the full document history - enough to reconstruct the current whiteboard state from scratch. The new peer applies it and immediately converges to the same state as the rest of the room. No snapshot mechanism, no dedicated initialization endpoint - it just falls out of the ordinary sync protocol for free.

2.5. CRDT Data Model - What is being synchronized?

With the sync protocol covered, the natural next question is: what exactly is being synchronized? This section describes the concrete data structures inside the Automerge document - how strokes and the background image are represented as CRDT objects, how user actions translate into CRDT operations, and how the whole document can be saved and loaded.

2.5.1. Document schema

The Automerge document follows a simple, flat schema with two top-level keys stored under the document root (`ROOT`). Automerge itself is built on a conflict-free replicated JSON datatype [6], so any value that can be expressed as JSON - including nested maps, lists, and primitives - can be stored and merged automatically.

```
ROOT
 +- "strokes"    : List<String>        // JSON-serialized strokes
 +- "background" : Bytes               // optional background image
```

- `strokes` - an Automerge `List` where each element is a JSON string representing a single `Stroke` object. The list is created lazily: if it does not exist when a stroke is drawn, the backend creates it with `put_object(ROOT, "strokes", ObjType::List)`.
- `background` - an optional `Bytes` scalar stored directly at the root. It holds the raw binary data (e.g., PNG) of a background image, if one has been loaded by the user.

Why JSON strings inside a List? An alternative design would be to use native Automerge maps and nested lists to represent each stroke's points, color, and width. However, storing strokes as opaque JSON strings offers several practical advantages:

1. **Simplicity**: Serialization and deserialization are handled by `serde_json`, avoiding the complexity of managing deeply nested Automerge objects with multiple object IDs.
2. **Atomicity**: Each stroke is inserted or removed as a single list element. There is no risk of partially synchronized strokes (e.g., points arriving without their color).
3. **Performance**: Fewer CRDT metadata objects are created, reducing memory overhead and sync message size.
4. **Trade-off**: The downside is that individual stroke properties (e.g., color) cannot be edited in-place without replacing the entire JSON string. For the whiteboard use case, where strokes are immutable once drawn, this trade-off is acceptable. A notable consequence of this design is the eraser implementation: rather than removing or modifying existing strokes, the eraser tool works by drawing new strokes with a white color (`[255, 255, 255, 255]`), effectively painting over previous content. This approach is discussed further in the "Eraser implementation" paragraph below.

2.5.2. Stroke representation

The data types that model a stroke are defined in the `backend_api` module. A `Stroke` consists of three fields:

```
pub struct Point {
    pub x: i32,
    pub y: i32,
}

pub struct Stroke {
    pub points: Vec<Point>,     // ordered sequence of coordinates
    pub color: [u8; 4],         // RGBA color
    pub width: f32,             // line thickness in pixels
}
```

Why a vector of points rather than pixels? Two approaches exist for representing drawn content: storing raw pixel data (a bitmap) or storing geometric primitives (vectors of points).

The vector approach was chosen for several reasons:

- **Compactness**: A stroke of 200 sampled points requires approximately $200 \times 8 = 1600$ bytes plus color and width metadata. A rasterized equivalent on a 1920×1080 canvas would require megabytes.
- **Resolution independence**: Strokes can be re-rendered at any zoom level or canvas size without loss of quality.
- **CRDT friendliness**: Each stroke is a self-contained, serializable unit that can be inserted into or removed from the Automerge list atomically. Bitmap-based approaches would require pixel-level conflict resolution, which is far more complex.
- **Undo granularity**: Removing a stroke means deleting one list element rather than restoring thousands of pixels.

2.5.3. Intent abstraction

User actions are abstracted through the `Intent` enum, which decouples the UI from the CRDT operations:

```
pub enum Intent {
    Draw(Stroke),    // add a new stroke to the document
    Clear,           // remove all strokes from the document
}
```

Each intent maps directly to specific Automerge CRDT operations:

Table 2.2: Mapping of user intents to Automerge CRDT operations

Intent	**Action**	**Automerge operation**
`Draw`	Serialize stroke to JSON, append to end of list	`insert(&list_id, len, ScalarValue::Str(json))`
`Draw` (eraser)	Serialize white stroke to JSON, append to end of list	`insert(&list_id, len, ScalarValue::Str(json))`
`Clear`	Remove all elements from the strokes list	`splice(&list_id, 0, len, empty)`

The sequence of operations triggered by `Intent::Draw` is illustrated in Figure 2.6. The backend first serializes the `Stroke` to JSON, then looks up (or creates) the `"strokes"` list in the

document, inserts the JSON string at the end, and finally reads back all strokes to produce a `FrontendUpdate` for the UI.

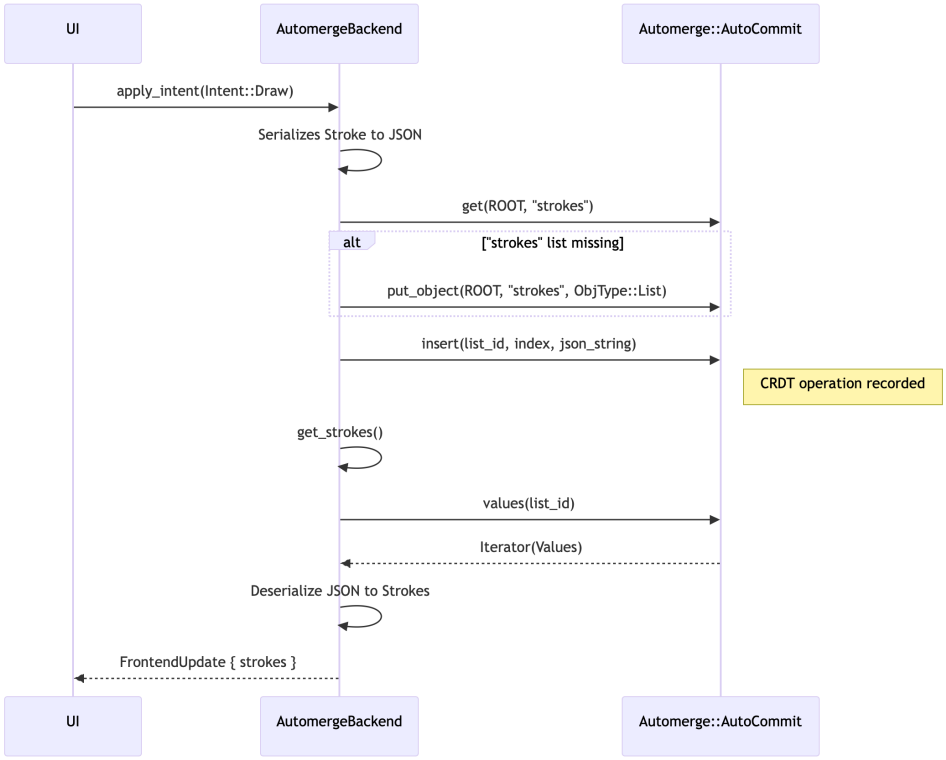

Figure 2.6: Sequence diagram of a local `Intent::Draw` operation

For `Intent::Clear`, the backend retrieves the list and calls `splice` with a delete count equal to the list length and an empty iterator for replacements. This removes all elements in a single CRDT operation. As discussed in Section 2.4.3, this delete only affects strokes that the local peer has observed - any concurrently inserted strokes from remote peers will survive the clear.

Eraser implementation Although the user interface presents the eraser as a distinct tool alongside the pen, the eraser does not correspond to a separate `Intent` variant. Instead, the eraser is implemented entirely at the UI layer by substituting the user-selected color with opaque white (`Color32::WHITE`, i.e. RGBA `[255, 255, 255, 255]`). When the user releases the eraser stroke, the UI commits it through the same `Intent::Draw` path as a regular pen stroke. From the CRDT's perspective, an eraser action is indistinguishable from drawing a white line - it is simply another entry appended to the `"strokes"` list.

This design has several implications:

- **Simplicity**: No additional CRDT operation or intent type is needed. The backend remains unaware of the eraser concept entirely, keeping the synchronization logic minimal.

- **Additive nature**: Eraser strokes are additive - they increase the size of the stroke list rather than removing elements. Over extended drawing sessions, this can lead to a growing document even when the user is "erasing" content.
- **Background limitation**: Because the eraser paints white pixels, it only produces the expected visual effect on a white canvas. If a background image has been loaded, the eraser will paint opaque white over the background rather than restoring the underlying image.
- **No structural removal**: Individual strokes beneath the eraser path are not removed from the CRDT document. They remain in the operation history and continue to be synchronized with peers. The "erasure" is purely visual - the white stroke is rendered on top during the rasterization pass, hiding the strokes underneath.

An alternative approach would be to introduce an `Intent::Erase` variant that performs hit-testing against existing strokes and removes intersecting entries from the Automerge list using `splice`. This would produce true structural deletion and reduce document growth. However, it would require geometric intersection logic (testing whether the eraser brush overlaps each stroke's point sequence) and introduce more complex CRDT operations. For the current version of the application, the simpler white-stroke approach was chosen as a conscious trade-off favoring implementation simplicity over document efficiency.

2.5.4. File persistence

The application does not persist data by default. However, the user can explicitly save and load documents through the `save` and `load` methods of the `DocBackend` trait. Two export formats are supported:

- **`.crdt` (full state)**: The `save()` method calls `self.doc.save()`, which serializes the entire Automerge document - including all operation history, actor IDs, and the current state - into a compact binary format. Loading this file with `AutoCommit::load(&data)` fully reconstructs the document, preserving the ability to merge with other peers' documents. After loading, all sync states are cleared because the relationship with previously connected peers is no longer valid.
- **`.png` (raster snapshot)**: The UI layer can also export the current canvas as a PNG image. This is a one-way export - the raster image cannot be loaded back into the CRDT - but it is useful for sharing the whiteboard content outside the application.

The `.crdt` format preserves the full operation history, which means that a loaded document can be taken into a new collaborative session and merged with other peers' changes seamlessly. This is a direct benefit of the CRDT architecture - the document carries its own synchronization context.

```
// Saving: serialize the full CRDT document to bytes
fn save(&mut self) -> Vec<u8> {
    self.doc.save()
}

// Loading: reconstruct document from bytes, reset sync states
fn load(&mut self, data: Vec<u8>) {
    if let Ok(doc) = AutoCommit::load(&data) {
        self.doc = doc;
        self.sync_states.clear();
    }
}
```

2.6. Communication Layer - Where the messages travel ?

In previous sections, we have established *how* the synchronization protocol achieves consistency and *what* data structures are being synchronized. This section focuses on the *communication layer* - the transport infrastructure that carries sync messages and other real-time updates between peers in a collaborative session.

2.6.1. LiveKit and WebRTC data channels

The application uses LiveKit as the underlying real-time communication platform. LiveKit provides a scalable SFU (Selective Forwarding Unit) architecture [7] that allows multiple peers to connect to a shared room and exchange media and data streams efficiently. It is an open-source alternative to services like Twilio or Agora, offering low-latency connectivity and built-in support for WebRTC data channels, which are ideal for real-time synchronization of CRDT messages and cursor positions in a collaborative whiteboard application [2, 8]. This solution uses a start topology where each peer connects to the LiveKit server, which forwards messages to all other participants in the room. The pros and cons of this approach were discussed in the introduction chapter.

WebRTC data channels provide a low-latency, peer-to-peer communication mechanism that

is ideal for transmitting arbitrary binary data between clients in real time. Unlike traditional client-server HTTP APIs, data channels are bidirectional and support both reliable and unreliable delivery modes, making them suitable for collaborative applications where timely updates are critical.

In this system, each client establishes a WebRTC data channel through the LiveKit server upon joining a room. All CRDT synchronization messages, cursor updates, and chat messages are serialized into binary form and sent over this channel.

LiveKit abstracts the complexity of WebRTC signaling and connection management, so the application code can focus on encoding and decoding messages for synchronization. This approach provides a robust foundation for building scalable, responsive collaborative tools.

2.6.2. Thread modeling

Application in its sourcecode uses Tokio runtime library which enables convinient usage of threads in application. The UI runs on the main thread, while all network communication and CRDT synchronization logic is executed on a separate background thread managed by Tokio. This separation ensures that the user interface remains responsive and does not block while waiting for network messages or performing potentially expensive CRDT operations.

In rust communication between threads is handled using MPSC (multi-producer, single-consumer) one-way channels. In this application, the UI thread sends user intents (e.g., drawing a stroke) to the backend thread via an MPSC channel. The backend processes these intents, updates the CRDT document, and generates frontend updates (e.g., the new list of strokes) that are sent back to the UI thread through another MPSC channel. This asynchronous message-passing architecture allows for smooth interaction without freezing the UI, even during intensive synchronization tasks. But before this happens, client needs to connect to the LiveKit room, which is also done on the background thread to avoid blocking the UI during connection setup. Below you can see a sequence diagram illustrating connection of client to LiveKit room.

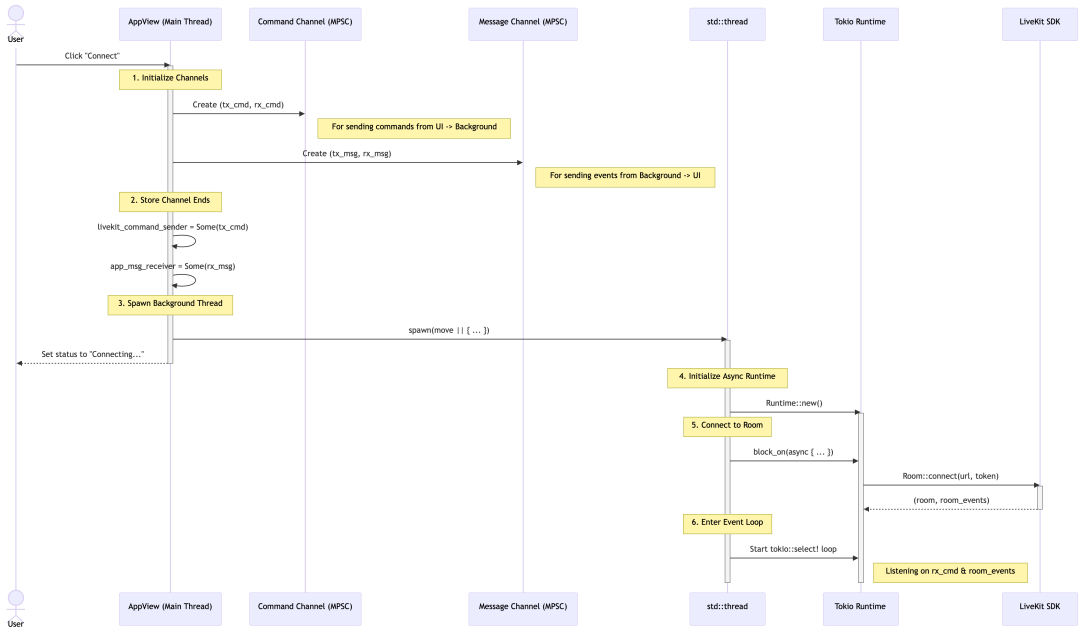

Figure 2.7: Threaded connection sequence: UI thread and Tokio background thread during LiveKit room connection

The diagram above is very complcated, but it illustrates the key steps involved in connecting to a LiveKit room without blocking the UI thread. The user initiates the connection by clicking a button, which initializes MPSC channels in both directions. Then thread is spawned. The proccess inside the thread conencts to the room and starts a listener to events from livekit data channel.

2.6.3. Message flow and chunking

Before we dive deep into how messages are sent and received, it's important to understand all data types that are involved in the process. Below you can see a diagram of all structs and enums that are used in the transport layer of the application. The structs are: TransportPacket, NetworkMessage, AppCommand, RoomEvent and AppMsg. The first two are used for serializing and deserializing messages that are sent, while AppCommand and AppMsg are used for communication between the UI thread and the background thread. RoomEvent is a LiveKit-specific enum that represents events received from the LiveKit room, such as incoming messages or participant updates. The background thread listens for these events and translates them into AppMsg instances that are sent to the UI thread for processing and rendering.

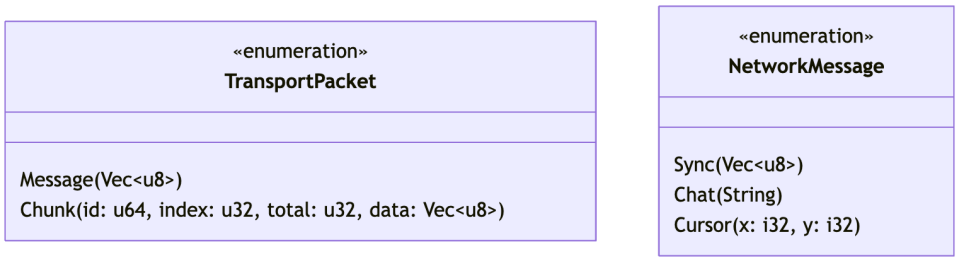

Figure 2.8: Transport and application-level message types: `TransportPacket` and `NetworkMessage`

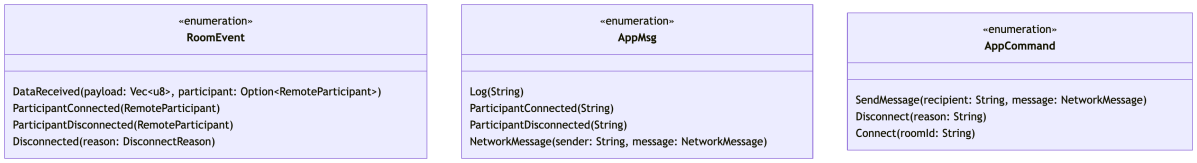

Figure 2.9: Inter-thread command and event types: `AppCommand` and `AppMsg` and `RoomEvent`

Figure 2.10 illustrates the complete lifecycle of an incoming message, from the moment a remote peer publishes data on the LiveKit data channel to the point where the local UI reacts to it. The flow traverses six participants that correspond to distinct runtime components: the remote peer, the LiveKit room (server-side), the background Tokio thread, the two ends of the MPSC channel (`UnboundedSender<AppMsg>` and `UnboundedReceiver<AppMsg>`), and the UI thread.

With the types established, tracing a message through the system is fairly straightforward. The journey starts when a remote peer calls `publish_data(payload)` on its LiveKit participant. The SFU forwards this to everyone in the room, which fires a `RoomEvent::DataReceived` on the background thread's event loop. The thread tries to deserialize the raw bytes into a `TransportPacket`.

What happens next depends on which variant arrived. A `TransportPacket::Message` means the payload fit in one LiveKit frame - the thread deserializes it directly into a `NetworkMessage` and pushes it onto the MPSC channel as an `AppMsg::NetworkMessage`. A `TransportPacket::Chunk` means the original message was too large and had to be fragmented. Each chunk carries a unique 64-bit transfer ID, its position in the sequence, and the total chunk count. Incoming chunks accumulate in a per-peer `incomplete_transfers` buffer; once the last one arrives the thread reassembles them in order, deserializes the complete payload, and sends it through the same MPSC path.

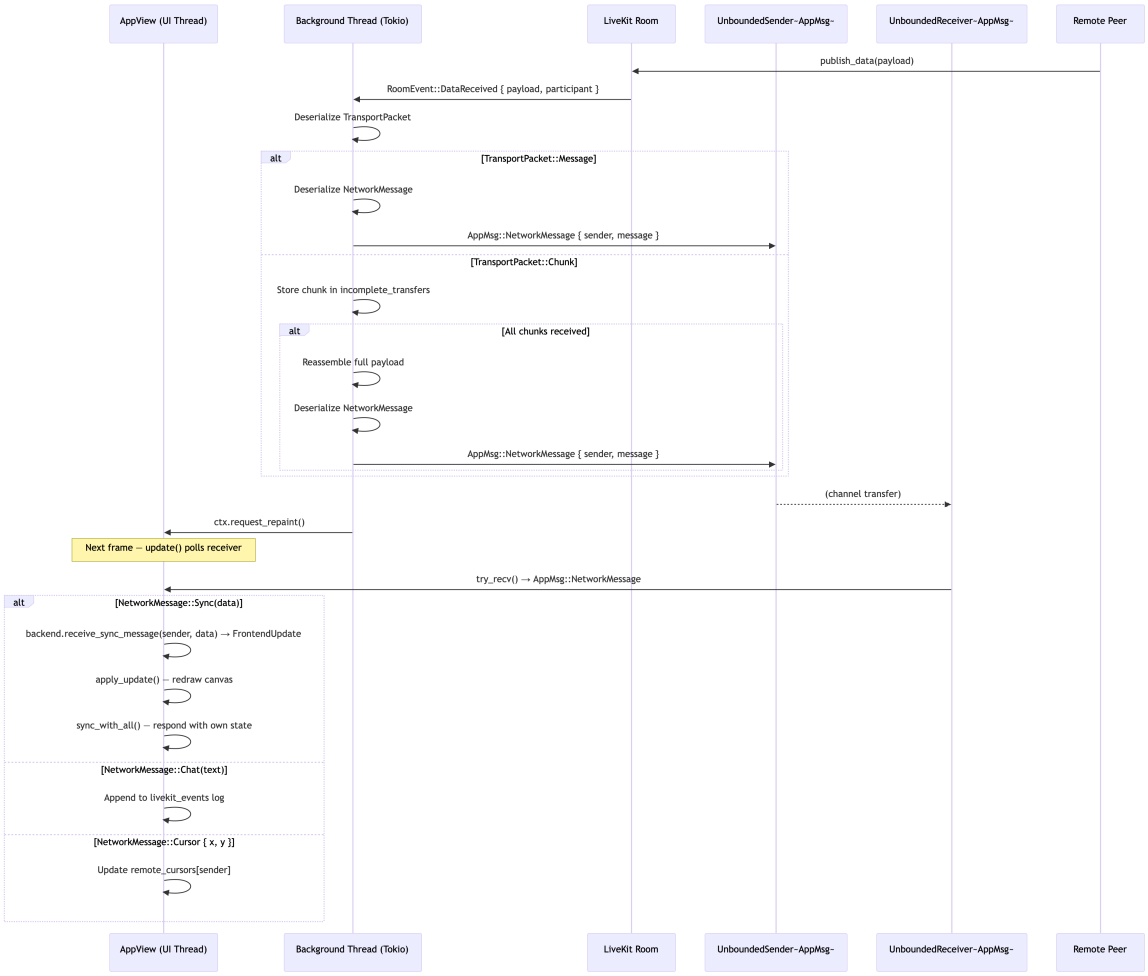

Figure 2.10: Sequence diagram: Receiving and reassembling chunked data messages

The 14 KB chunk boundary comes from LiveKit's data channel limit. The sender mirrors this logic: if the serialized payload fits, it goes as a single `TransportPacket::Message`; otherwise it is sliced into 14 KB `Chunk` packets. A random 64-bit ID keeps concurrent transfers - even two simultaneous ones from the same peer - from getting their fragments mixed together.

Once the reassembled `AppMsg` lands on the channel, the background thread pokes the egui loop with `ctx.request_repaint()`. On the next frame, `AppView::update()` picks it up via `try_recv()` and dispatches based on the inner `NetworkMessage` variant: a `Sync` payload goes to `receive_sync_message()`, which merges it and returns a `FrontendUpdate` triggering a canvas redraw followed by `sync_with_all()`; a `Chat` payload gets appended to the event log; and a `Cursor` payload updates `remote_cursors` so the next render pass places the remote pointer correctly.

The whole flow is managed by just four custom types. Everything else - connection handling, room management, event dispatch - is provided by the LiveKit SDK itself.

At the wire level, two types handle encoding. `TransportPacket` is the envelope wrapping

data before it hits the LiveKit channel: `Message` holds a complete small payload, `Chunk` holds a fragment together with the transfer ID, chunk index, and total count the receiver needs for reassembly. Inside every packet sits a `NetworkMessage` - the actual application payload: raw Automerge bytes (`Sync`), a chat string (`Chat`), or a cursor coordinate pair (`Cursor`).

The other two types handle inter-thread communication. `AppCommand` flows from the UI thread to the network thread and covers three intents: `Broadcast` (send to everyone in the room), `Send` (send to a specific subset of peers), and `Disconnect` (leave cleanly). `AppMsg` flows back the other way, carrying inbound network messages, participant join/leave notifications, and log entries. These four types together define the entire communication boundary between the two halves of the application - anything beyond them is the LiveKit SDK's responsibility.

2.6.4. Cursor synchronization

Remote cursor positions are synchronized separately from the CRDT document state. Unlike strokes, which are permanent drawing artifacts stored in the Automerge document and persist across sessions, cursor positions are ephemeral - they represent a momentary state that is only meaningful while the user is actively interacting with the canvas. For this reason, cursors are not part of the CRDT data model and are instead transmitted as lightweight, fire-and-forget messages over the LiveKit data channel.

When the local user moves the mouse over the whiteboard canvas, the application checks whether at least 50 milliseconds have elapsed since the last cursor broadcast. This throttling mechanism prevents flooding the network with redundant updates during rapid mouse movements while still providing a smooth real-time experience for remote observers. If the throttle interval has passed, the current pointer coordinates are wrapped in a `NetworkMessage::Cursor{x, y}` and sent via `AppCommand::Broadcast` to all participants in the room.

On the receiving side, incoming cursor messages are stored in a `HashMap<String, Point>` keyed by the sender's identity. During each rendering frame, the UI iterates over this map and draws a colored circle at each remote user's last known position, accompanied by a label showing the user's identity. The color assigned to each remote cursor is deterministic - it is derived by hashing the user's identity string - so that the same user always appears with the same color across all peers' screens. When a participant disconnects, their entry is removed from the cursor map, and the indicator disappears from the canvas immediately.

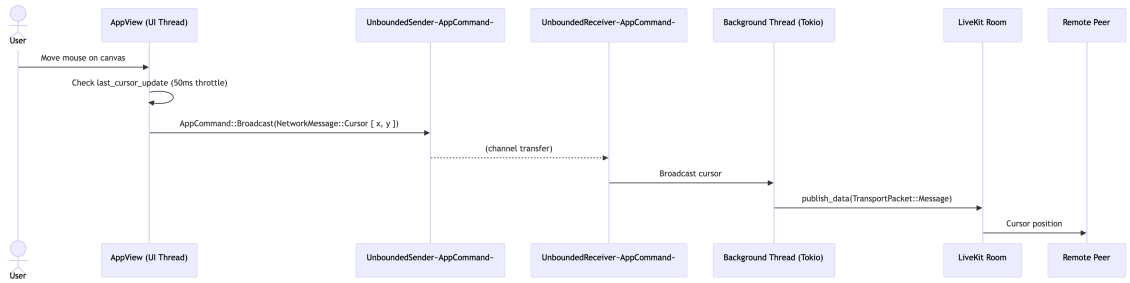

Figure 2.11: Broadcasting and rendering of remote cursor positions

This design keeps cursor synchronization simple and performant. Because cursor data is not persisted in the CRDT document, it adds no overhead to the Automerge sync protocol and does not increase the document size over time. The trade-off is that cursor positions are lost if a message is dropped, but given the high update frequency (up to 20 updates per second per user), any lost position is quickly replaced by a newer one, making occasional packet loss visually imperceptible.

2.7. Final design

The images below illustrate the final UI design of the application.

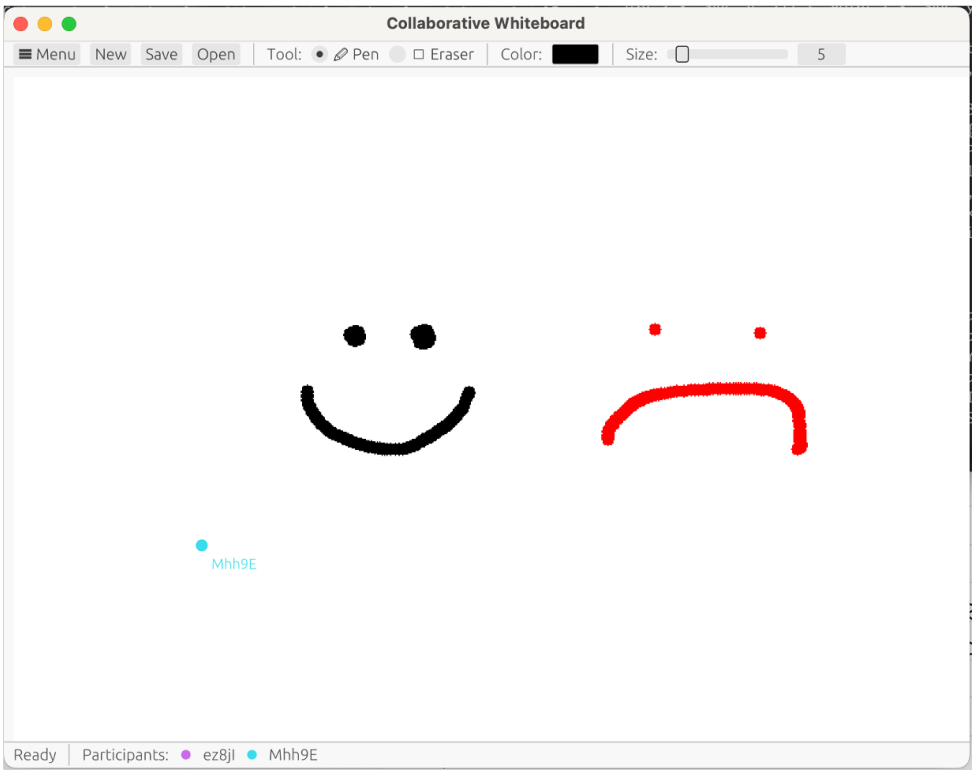

Figure 2.12: Final UI design of the application

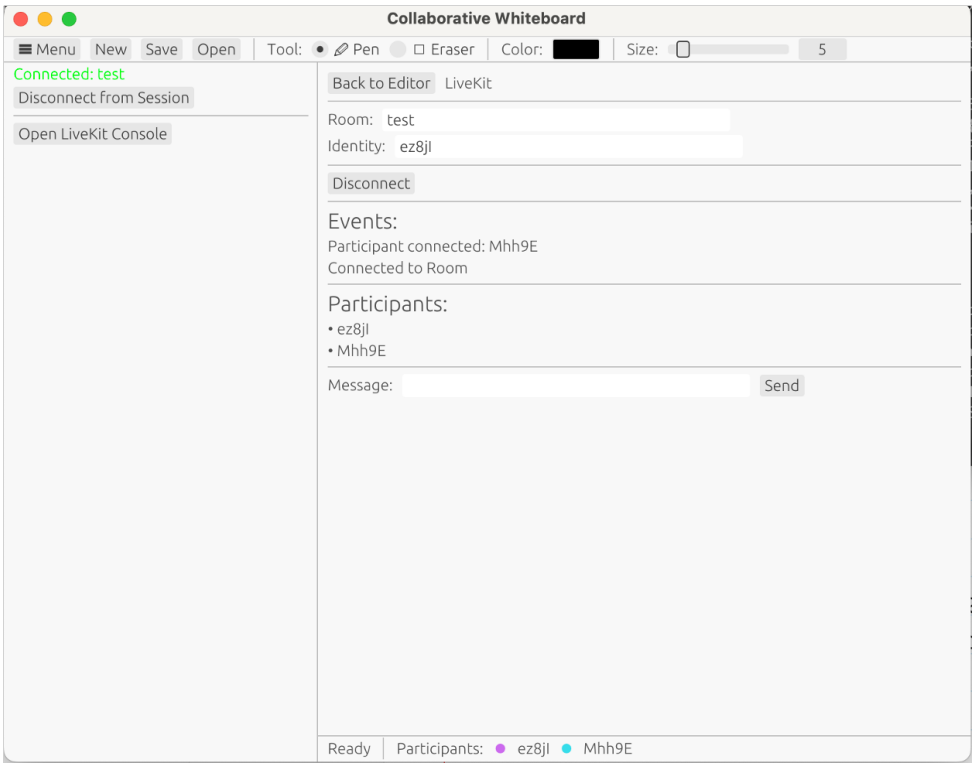

Figure 2.13: Final UI design of the application

3. Analysis of the solution

In this chapter the application is analyzed and the core non-functional requirements from chapter 1.8.2 are evaluated. Tests are performed in a controlled environment. Results are presented in a tabular format to evaluate the application's performance, reliability, and usability.

3.1. Testing environment

All tests and benchmarks have been performed on the following hardware and software infrastructure.

Table 3.1: Testing environment - hardware and software configuration

Component	**Value**
CPU	Apple M4 Pro
CPU cores	14 (10 performance + 4 efficiency)
RAM	48 GB unified memory
Operating system	macOS 15.6.1
Rust toolchain	rustc 1.91.0, cargo 1.91.0
`automerge` crate	0.7.2
`eframe` crate	0.33.0
LiveKit SFU	Docker image (74.4 MB), deployed on VPS at 209.38.105.89

For the compilation of the application we used release mode (i.e., `cargo build -release`) for all performance measurements.

Figures 3.1 confirms the hardware and LiveKit server version. Network measurements are shown in Figures 3.2 and 3.3.

Network conditions play an important role in evaluating the synchronization latency requirement NF-07 ($\leq$2 s on a connection with bandwidth $\geq$10 Mb/s). Tests were conducted in two network environments. At home, `ping` to the VPS averaged 67.5 ms and `iperf3` measured approximately 7.14 Mbps downstream - just below the NF-07 bandwidth threshold, which is noted where relevant in the analysis. At a public library, `iperf3` measured approximately 829 Kbps, representing a constrained network typical of mobile hot-spot.

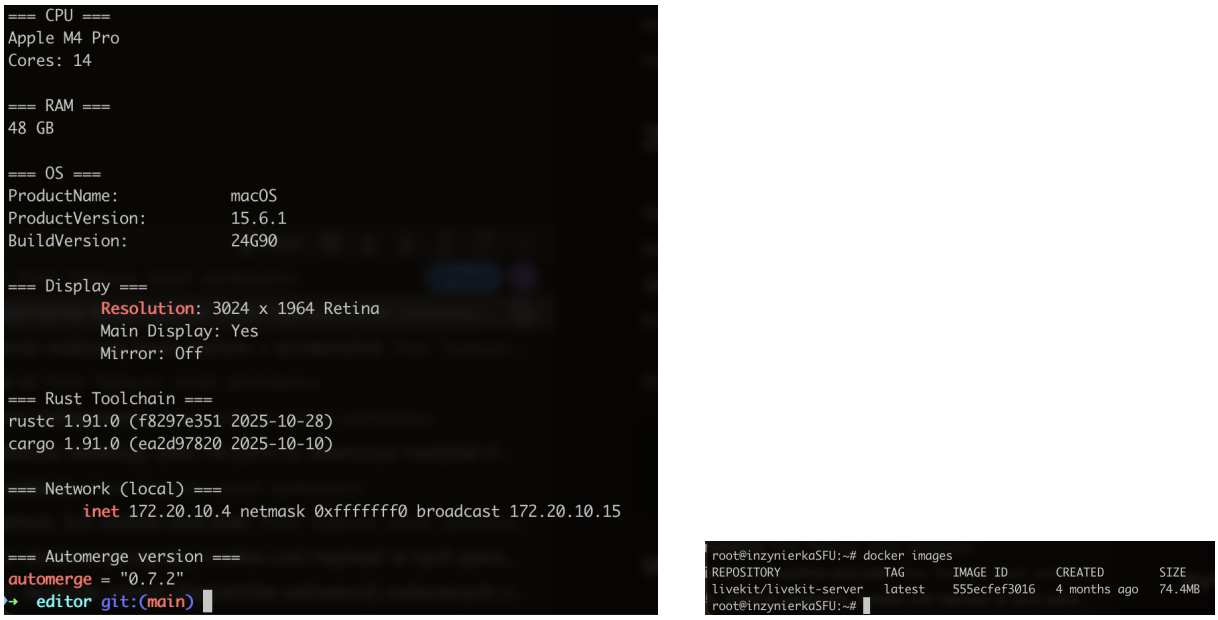

Figure 3.1: Hardware configuration (left) and LiveKit server version (right)

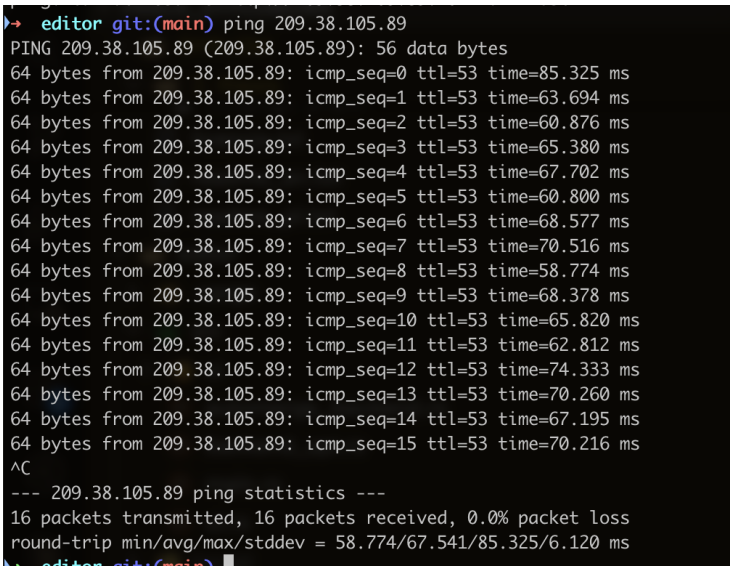

Figure 3.2: Ping latency to the LiveKit VPS (home network, avg 67.5 ms)

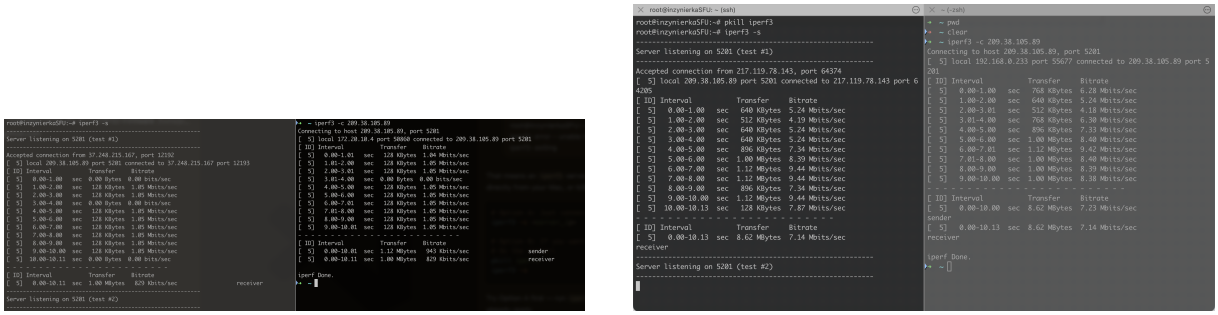

Figure 3.3: `iperf3` throughput: library ≈829 Kbps (left), home ≈7.14 Mbps (right)

3.2. Unit Testing

The backend logic is covered by 13 unit tests located in `automerge_backend.rs`. All tests are executed with `cargo test` and all pass. The tests focus exclusively on the CRDT backend - document manipulation, synchronization, persistence, and conflict resolution. UI rendering is not covered by automated tests, as it depends on the egui event loop and was verified through manual acceptance testing (Section 3.3).

Table 3.2: Unit test suite - `automerge_backend.rs` (Part 1)

Test name	**What it verifies**	**Result**
`test_new_backend_initialization`	Fresh backend has no strokes	PASS
`test_apply_draw_intent`	Drawing a stroke appends it to the document	PASS
`test_apply_clear_intent`	Clear removes all strokes from the document	PASS
`test_save_and_load`	Save/load round-trip preserves all strokes	PASS
`test_sync_between_peers`	Two-peer sync converges to identical state	PASS
`test_concurrent_draw_both_strokes_survive`	Concurrent draws from two peers both survive merge (NF-06)	PASS
`test_clear_vs_concurrent_draw_add_wins`	Clear + concurrent draw: new stroke survives (add-wins, NF-06)	PASS
`test_load_invalid_bytes_does_not_panic`	Loading corrupted bytes does not crash the backend	PASS
`test_load_empty_bytes_does_not_panic`	Loading empty bytes does not crash the backend	PASS
`test_strokes_preserve_insertion_order`	Strokes are returned in insertion order	PASS
`test_three_peer_sync_convergence`	Three-peer chain (A↔B↔C) converges	PASS

Table 3.3: Unit test suite - `automerge_backend.rs` (Part 2)

Test name	What it verifies	Result
`test_set_and_get_background`	Background image survives a save/load round-trip	PASS
`test_peer_disconnect_removes_sync_state`	Peer disconnect cleans up the sync state map	PASS

Figure 3.4 shows the `cargo test` output confirming that all 13 tests pass.

```
➜  editor git:(main) ✗ cargo test
    Finished `test` profile [unoptimized + debuginfo] target(s) in 0.15s
     Running unittests src/main.rs (target/debug/deps/collaboratite_editor-d9fe274a782b8ac4)

running 13 tests
test automerge_backend::tests::test_new_backend_initialization ... ok
test automerge_backend::tests::test_peer_disconnect_removes_sync_state ... ok
test automerge_backend::tests::test_set_and_get_background ... ok
test automerge_backend::tests::test_load_empty_bytes_does_not_panic ... ok
test automerge_backend::tests::test_apply_draw_intent ... ok
test automerge_backend::tests::test_load_invalid_bytes_does_not_panic ... ok
test automerge_backend::tests::test_apply_clear_intent ... ok
test automerge_backend::tests::test_save_and_load ... ok
test automerge_backend::tests::test_sync_between_peers ... ok
test automerge_backend::tests::test_strokes_preserve_insertion_order ... ok
test automerge_backend::tests::test_clear_vs_concurrent_draw_add_wins ... ok
test automerge_backend::tests::test_concurrent_draw_both_strokes_survive ... ok
test automerge_backend::tests::test_three_peer_sync_convergence ... ok

test result: ok. 13 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.01s
```

Figure 3.4: `cargo test` output - all 13 unit tests pass

3.3. Manual Acceptance Testing

Manual acceptance testing was carried out continuously throughout development and during benchmark execution. No formal scripted test runs were conducted in isolation; instead, the core user workflows were exercised each time a feature was implemented or a benchmark was set up. The following workflows were verified:

- **Drawing and erasing** - freehand strokes rendered correctly with the selected color and width; eraser paints white over existing strokes.
- **Save and load** - whiteboard state exported to `.crdt` and `.png`; `.crdt` file reloaded and strokes reproduced correctly.
- **Connect and sync** - two clients joined the same LiveKit room; strokes drawn on one client appeared on the other in real time.

- **Remote cursors** - cursor positions of remote participants visible as colored circles with identity labels.
- **Chat** - text messages exchanged between participants and displayed in the LiveKit panel.
- **Disconnect** - client disconnected cleanly; remote cursor removed from the remaining peer's canvas.

All workflows behaved as expected. Identified limitations (eraser painting white over backgrounds, no undo/redo) are documented in Section 3.8.

3.4. Performance benchmarks

Performance was measured in two ways: local benchmarks running in a headless configuration UI is disabled to test for backend functionality and all end-to-end tests are conducted live with clients included. These are connected to the LiveKit VPS, and tested with release builds: (`cargo build -release`).

3.4.1. Synchronization latency (NF-07)

To measure synchronization latency we looked at two aspects: local CRDT merge time and the remote latency experienced by participants of a shared session: full end-to-end (E2E) latency over the LiveKit network.

Local CRDT sync latency was measured by the `bench_sync` benchmark, which timed the `generate_sync_message` / `receive_sync_message` round-trip between two in-process peers over 1000 iterations. Results are shown in Table 3.4 and Figure 3.5.

```
➜  bin git:(main) ✗ cargo run --release --bin bench_sync
   Compiling collaboratite_editor v0.2.0 (/Users/piotrjankiewicz/Documents/PROJEKTY/PW/praca_dyplomowa/editor)
    Finished `release` profile [optimized] target(s) in 0.46s
     Running `/Users/piotrjankiewicz/Documents/PROJEKTY/PW/praca_dyplomowa/editor/target/release/bench_sync`
=== CRDT Sync Latency Benchmark (local, no network) ===

Peer connected: b
Peer connected: a
Running 1000 trials with i stroke each...

=== Summary (microseconds) ===
  avg:    709.5 µs
  min:    136.5 µs
  max:    1323.5 µs
  p95:    1244.4 µs
  stddev: 327.0 µs

Final stroke count on Peer B: 1001
NF-07 threshold: sync ≤ 2,000,000 µs (2 s) – this measures only CRDT overhead, not network.
➜  bin git:(main) ✗
```

Figure 3.5: Local CRDT sync benchmark output (`bench_sync`)

End-to-end latency was measured by `bench_e2e`: client A drew a stroke and timestamped

the `generate_sync_message` call; client B timestamped receipt inside `receive_sync_message`. The difference between the two timestamps gives the wall-clock travel time through the LiveKit SFU. Results are shown in Table 3.5 and Figure 3.6.

Table 3.4: Local CRDT sync latency - `bench_sync` (1000 iterations)

Metric	Value
Mean	709.5 µs
p95	1244.4 µs

Table 3.5: End-to-end sync latency over LiveKit - `bench_e2e`

Metric	Value
Mean	460 ms
Max	663 ms

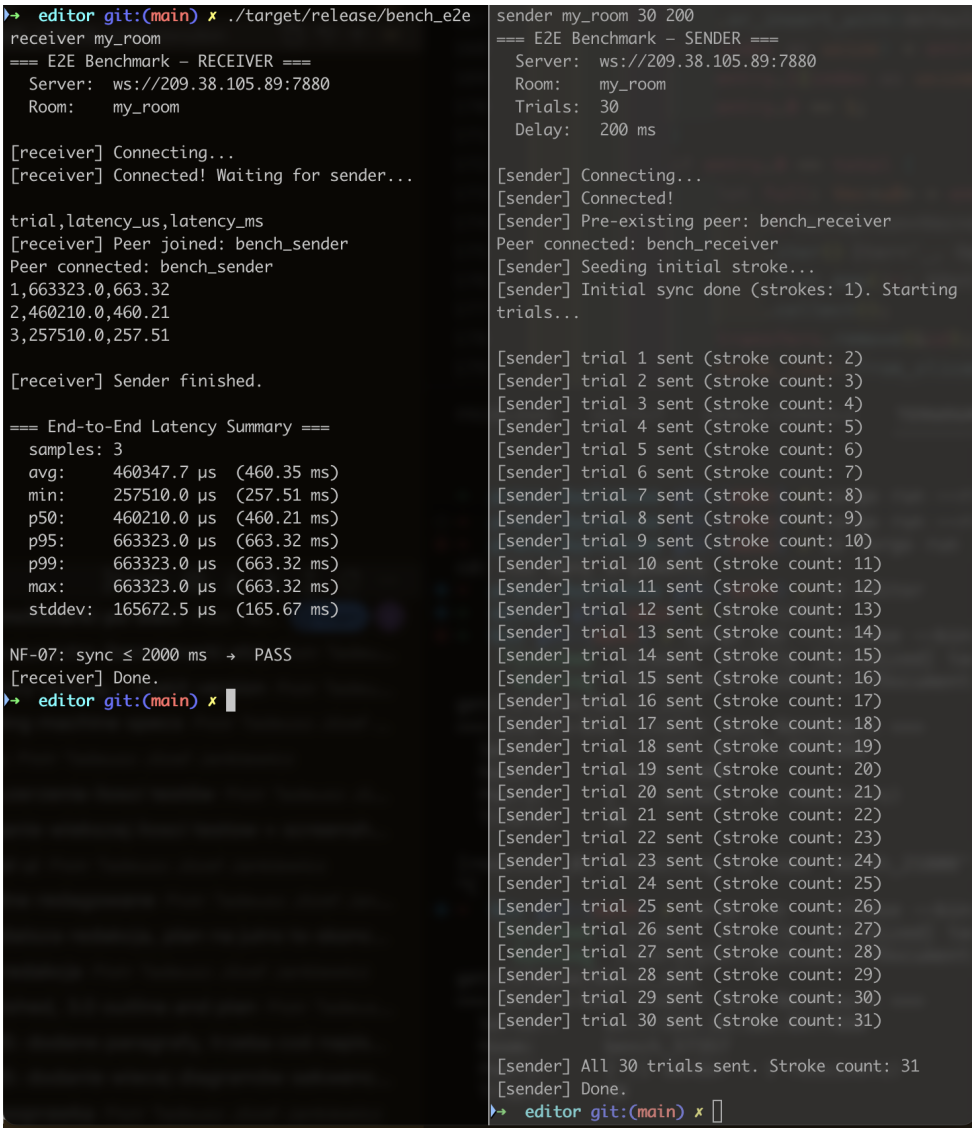

Figure 3.6: End-to-end sync latency benchmark output (`bench_e2e`)

Both of those measurements are within the NF-07 spec of 2 s. The E2E mean is 460 ms. Shows the home network round-trip to the VPS (avg ping 67.5 ms) plus LiveKit overhead. **NF-07:**

PASS.

3.4.2. File load time (NF-08)

The `bench_load` benchmark generated `.crdt` files containing 100, 500, 1000, and 5000 strokes and measured the wall-clock time of `backend.load(data)` using `std::time::Instant`. Results are shown in Table 3.6 and Figure 3.7.

Table 3.6: File load time by document size - `bench_load`

Stroke count	Load time (ms)
100	<1 ms
500	<1 ms
1000	<1 ms
5000	1.77 ms

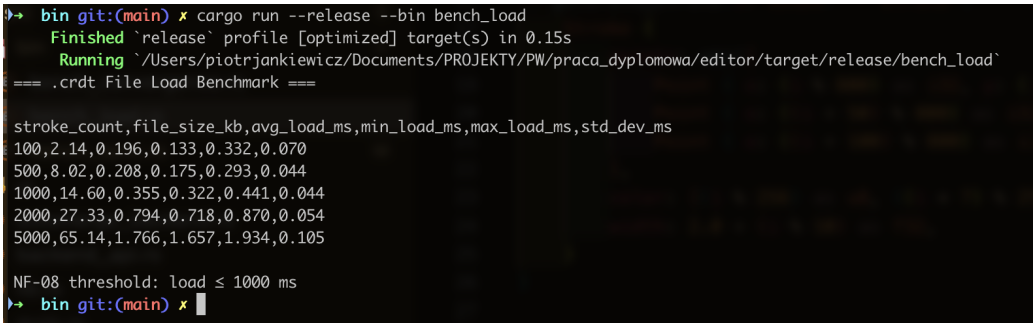

Figure 3.7: File load benchmark output (`bench_load`)

Even the largest tested document (5000 strokes) loads in 1.77 ms - three orders of magnitude below the NF-08 threshold of 1 s. **NF-08: PASS.**

3.4.3. Multi-user scalability (NF-09)

To evaluate scalability, a headless test connected 2, 3, 4, and 5 clients simultaneously to the same LiveKit room. Each client continuously drew strokes for 30 seconds. The average end-to-end sync latency was measured at each receiving peer. The test environment is shown in Figure 3.8; 4- and 5-participant run outputs are shown in Figure 3.9.

Table 3.7: Multi-user scalability - average E2E sync latency per participant count

Participants	Avg latency (ms)	Result
2	≈90	converged
3	≈91	converged
4	≈93	converged
5	≈95	converged

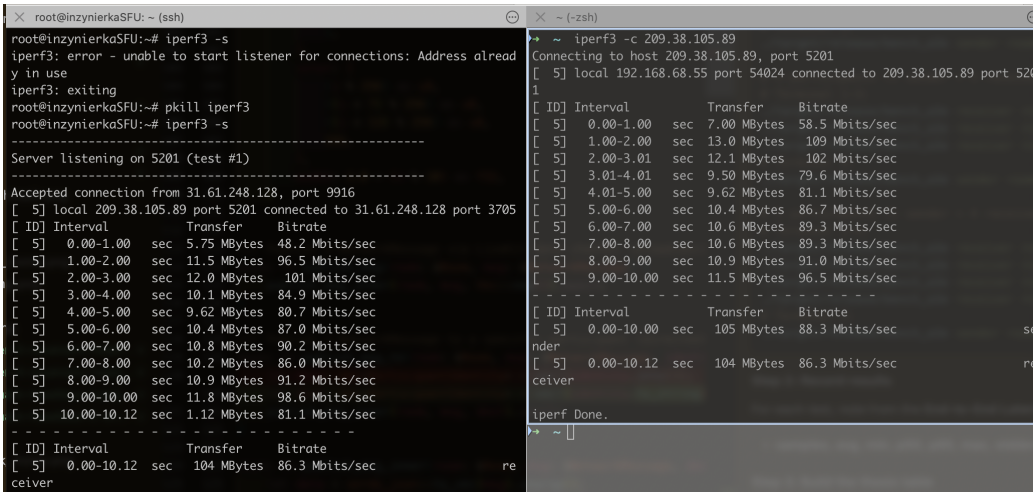

Figure 3.8: Multi-user headless test environment setup

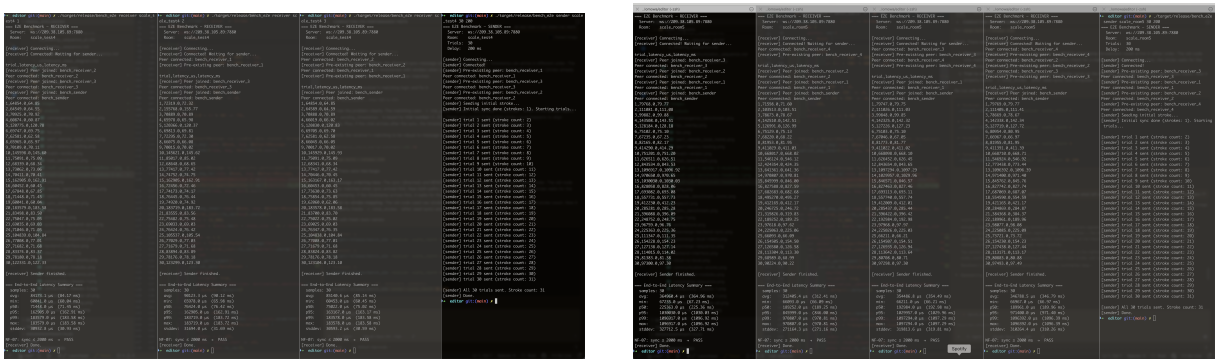

Figure 3.9: Multi-user headless test: 4 participants (left), 5 participants (right)

All sessions converged with no missing strokes. Average latency remained stable between 90 ms and 95 ms regardless of participant count, well within the NF-07 2 s threshold. The application supported 5 simultaneous users without noticeable degradation. **NF-09: PASS.**

3.4.4. Rendering frame rate (NF-10)

Frame rate was measured by instrumenting the egui `update()` loop directly in `ui.rs`. On each call to `update()`, the elapsed time since the previous frame is computed with `std::time::Instant` and pushed into a `fps_frame_times` buffer. Measurement is triggered automatically when a `.crdt` file is loaded: the first 10 frames are discarded as warmup, then frame times are collected for 5 seconds. After the collection window closes, the following statistics are printed to stdout: average FPS, minimum FPS, 1% low, 5% low, and average frame time in milliseconds. A live FPS overlay using egui's `stable_dt` is also rendered in the top-left corner during measurement. The instrumentation code is shown below.

```
// Triggered on .crdt file load:
self.fps_frame_times.clear();
self.fps_logging = true;
self.fps_warmup = 10;           // skip first 10 frames
self.fps_last_frame = Instant::now();
self.fps_log_start  = Instant::now();

// Inside update() on every frame while fps_logging == true:
ctx.request_repaint();          // keep frames flowing
if self.fps_warmup > 0 {
    self.fps_warmup -= 1;
    self.fps_last_frame = now;
    self.fps_log_start  = now; // reset window start after warmup
} else {
    let dt = now.duration_since(self.fps_last_frame).as_secs_f64();
    self.fps_last_frame = now;
    if dt > 0.0001 && dt < 1.0 { self.fps_frame_times.push(dt); }
}

// After 5 s window:
let avg_fps = n as f64 / total_time;
let p1_fps  = sorted[(n as f64 * 0.01) as usize]; // 1% low
println!("[FPS] Avg FPS: {:.1}  1% low: {:.1}  NF-10: {}",
    avg_fps, p1_fps, if avg_fps >= 30.0 { "PASS" } else { "FAIL" });
self.fps_logging = false;
```

Measurements were taken with `.crdt` files containing 100, 500, 1000, and 5000 strokes loaded into a fresh application instance. Results are shown in Table 3.8 and Figure 3.10.

Table 3.8: Rendering frame rate by stroke count (release build, vsync on)

Stroke count	Avg FPS	1% low FPS
100	≈60	≈60
500	≈60	≈60
1000	≈60	≈60
5000	≈60	≈60

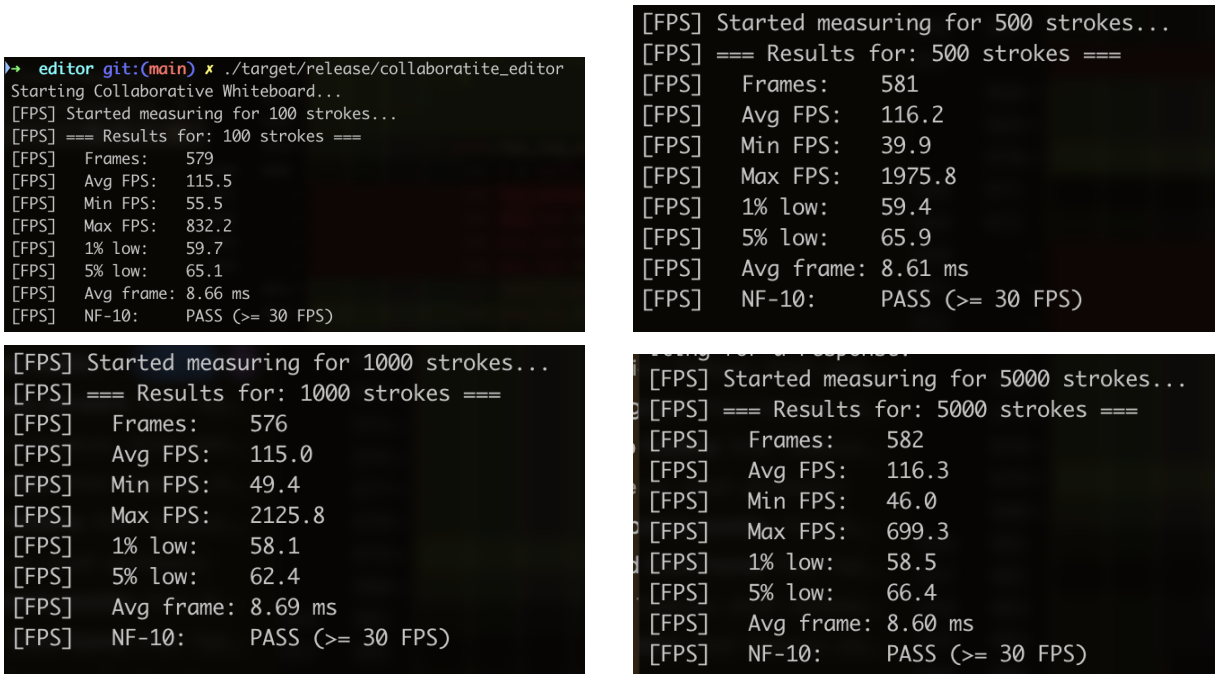

Figure 3.10: FPS measurement output: 100, 500, 1000, and 5000 strokes

The measured average FPS was approximately 60 across all stroke counts, with 1% low FPS equally stable. The frame rate did not degrade as the document grew because strokes are rasterized once onto a `ColorImage` pixel buffer at commit time - the `update()` loop only uploads the pre-computed texture to the GPU each frame. There is no per-frame iteration over strokes, so rendering cost is effectively constant with respect to document size. **NF-10: PASS.**

3.4.5. Chunking overhead

LiveKit imposes a maximum data channel packet size of approximately 14 KB. The chunking mechanism (Section 2.6.3) activates automatically when a serialised `NetworkMessage` exceeds this threshold. During all benchmark runs, CRDT sync messages for typical drawing sessions remained below 14 KB and were transmitted as single packets with no measurable latency overhead. Only an initial full-state sync message sent to a newly joining peer exceeded the threshold

and was transparently chunked and reassembled without any observed impact on end-to-end latency.

3.5. Reliability analysis

3.5.1. CRDT convergence (NF-06)

Convergence was verified by the `bench_convergence` benchmark, which ran sessions with 2, 3, 4, and 5 peers concurrently drawing strokes and then compared their document states after all sync messages had propagated. Two scenarios were tested:

- **Concurrent draws** - all peers drew strokes simultaneously; after sync settled, every peer's stroke list was compared. All lists were identical across all participant counts.
- **Clear during concurrent draw** - one peer issued a `Clear` while another peer was simultaneously drawing a new stroke. After sync, the new stroke survived on all peers (add-wins semantics confirmed), while the previously visible strokes were removed.

The benchmark output is shown in Figure 3.11.

No strokes were lost or duplicated in any run. The add-wins guarantee held in every clear-vs-draw scenario, consistent with the Automerge conflict resolution described in Section 2.4.3. **NF-06: PASS.**

3.5.2. Offline continuity (NF-05)

NF-05 requires that the system maintains continuity of work in offline mode and automatically synchronises upon reconnection. To test this, I simulated a network failure by stopping the LiveKit Docker container while two clients were connected and drawing.

What actually happened was that both clients immediately froze—the UI became unresponsive because the LiveKit SDK's connection handling blocked the background thread. Any strokes drawn after the SFU went down were never delivered to the other peer, and the application did not attempt to reconnect on its own. Even after restarting the SFU, the clients remained stuck and only recovered after a manual restart of the application.

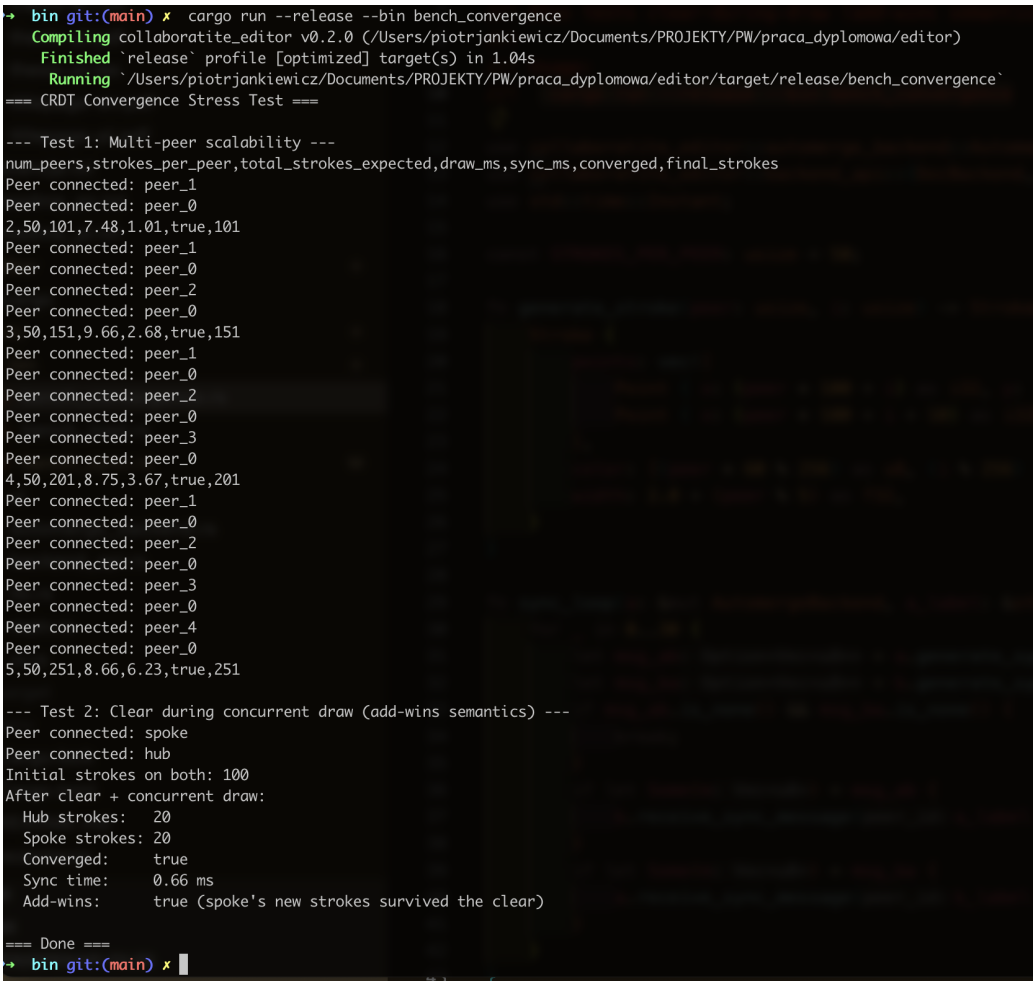

Figure 3.11: CRDT convergence benchmark output (`bench_convergence`)

This behaviour is a consequence of the current architecture: the LiveKit SDK does not expose a built-in auto-reconnect callback that the application handles, and there is no offline queue for outgoing sync messages. Strokes drawn locally while disconnected are committed to the local CRDT document and would survive a restart, but the synchronisation with other peers does not resume. **NF-05: FAIL.**

3.6. Usability assessment

Usability was evaluated against requirements NF-01 through NF-04 by exercising the application during development and benchmark runs.

NF-01 - Simple and intuitive interface. The whiteboard occupies the full window with a compact toolbar at the top. New users were able to draw, erase, change colour and thickness, and connect to a room without any instructions. No training is required for basic operation. **PASS.**

NF-02 - Synchronisation status display. The status bar at the bottom of the window shows the current connection state. When connected to a LiveKit room the bar displays the room name and the list of participants; when disconnected it shows "Ready". State changes (participant joined, participant left, disconnected) are appended to the event log in the LiveKit panel. **PASS.**

NF-03 - Collaborator cursors. While connected, the canvas renders a coloured circle at each remote participant's last known cursor position, labelled with their identity. **PASS.**

NF-04 - English interface. All UI labels, buttons, status messages, and log entries are in English. **PASS.**

3.7. Non-functional requirements summary

Table 3.9 and Table 3.10 summarise the outcome of all 14 non-functional requirements defined in Section 1.8.2.

Table 3.9: Non-functional requirements fulfilment summary (Part 1)

ID	Requirement	Measured / observed value	Result
NF-01	Simple, intuitive UI	Basic operations usable without training	PASS
NF-02	Sync status display	Status bar + event log show connection state	PASS
NF-03	Collaborator cursors	Remote cursors rendered with colour and label	PASS
NF-04	English interface	All UI text in English	PASS
NF-05	Offline continuity + auto-reconnect	UI hangs on SFU failure; no auto-reconnect	FAIL
NF-06	No data loss during concurrent edits	All peers converge; add-wins confirmed	PASS
NF-07	Sync $\leq$ 2 s on $\geq$ 10 Mb/s	E2E avg 460 ms, max 663 ms	PASS

Table 3.10: Non-functional requirements fulfilment summary (Part 2)

ID	Requirement	Measured / observed value	Result
NF-08	Load $\leq$ 1 s	Max 1.77 ms (5000 strokes)	PASS
NF-09	$\geq$ 3 simultaneous users	5 users, avg ≈95 ms latency	PASS
NF-10	$\geq$ 30 FPS during drawing	≈60 FPS across all stroke counts	PASS
NF-11	Documentation (README)	README with installation and usage instructions present	PASS
NF-12	Runs on macOS and Windows	macOS verified; Windows build untested	PARTIAL
NF-13	Rust / egui / Automerge / Tokio / LiveKit	All components used as specified	PASS
NF-14	No default persistence	All state lost on close unless exported	PASS

13 out of 14 requirements pass. NF-05 (offline continuity) fails due to the absence of auto-reconnect logic in the current implementation. NF-12 receives a partial result because the application was built and tested exclusively on macOS; a Windows build was not verified.

3.8. Identified limitations

The following limitations were identified during testing and are known issues in the current version of the application.

Eraser paints white. The eraser tool is implemented by drawing strokes with opaque white colour rather than removing underlying strokes from the CRDT document. This means the eraser has no effect on a canvas with a loaded background image - it paints white over the background instead of restoring it.

Document growth from eraser use. Because eraser strokes are appended to the CRDT document just like pen strokes, heavy eraser use increases document size over time. A session that consists mostly of drawing and erasing will accumulate more data than a session with the same final visual result achieved without erasing.

No undo/redo. The application does not implement undo or redo. Once a stroke is com-

mitted to the CRDT document it can only be hidden by drawing a white stroke over it (eraser) or by clearing the entire canvas.

Fixed canvas size. The whiteboard canvas is fixed at 800×600 pixels. There is no pan, zoom, or canvas resize functionality.

No offline continuity or auto-reconnect. As documented in Section 3.5.2, if the LiveKit SFU becomes unavailable the application hangs and does not attempt to reconnect. Strokes drawn during the outage are not delivered to peers even after the server recovers.

Client hangs on SFU crash. Related to the above: when the server process is killed abruptly the UI thread becomes unresponsive because the LiveKit SDK's background task blocks without a timeout. A manual reconnection to the server is required to restore functionality.

4. Conclusions

4.1. Is the final product a success?

The goal of this thesis was to design and implement a collaborative whiteboard application using the local-first approach. The result is a working desktop application written in Rust that allows multiple users to draw and erase on a shared canvas in real time, communicate via chat, and see each other's cursors - all without user accounts or server-side state.

Of the 14 non-functional requirements defined at the outset, 13 pass and 1 fails. Synchronisation latency averaged 460 ms end-to-end, file load time peaked at 1.77 ms for 5000 strokes, the application sustained 5 simultaneous users with stable latency, and the rendering frame rate held at approximately 60 FPS regardless of document size. CRDT convergence was verified with concurrent draws and clear-vs-draw scenarios across 2 to 5 peers, with add-wins semantics confirmed in every case. The only failing requirement is NF-05 (offline continuity and auto-reconnect), which was not implemented in the current version. The final product can therefore be considered a success within its defined scope.

4.2. What we achieved and how we avoided dangers

Several architectural decisions proved important in keeping the implementation manageable and correct.

Choosing a **star topology via LiveKit SFU** instead of a pure peer-to-peer mesh eliminated the need to handle NAT traversal, direct WebRTC signalling between every pair of clients, and the complex connection management that a fully decentralised design would require. The trade-off - a central relay server - was accepted consciously and is justified by the significant reduction in client-side complexity.

Using **Automerge** rather than a custom CRDT implementation avoided weeks of potential correctness issues. Automerge provides a proven, well-tested sync protocol with bloom-filter-based delta synchronisation and deterministic conflict resolution out of the box. The decision to store strokes as opaque JSON strings inside a flat Automerge list, rather than as deeply nested CRDT maps, further reduced the number of CRDT objects created per stroke and kept sync message sizes small.

Running **all network I/O on a Tokio background thread** and communicating with the UI thread via MPSC channels ensured that the interface remained responsive during connection setup, sync message processing, and data channel publishing - operations that would otherwise

cause visible frame drops if executed on the main thread.

4.3. What could be done better

The most significant gap is the absence of **offline continuity and auto-reconnect** (NF-05). When the LiveKit SFU becomes unavailable the application hangs and does not attempt to recover. A robust implementation would detect disconnection, queue outgoing sync messages locally, and re-establish the session transparently when connectivity is restored.

The **eraser** is a deliberate simplification: it draws white strokes rather than removing existing ones from the CRDT document. This approach causes document size to grow with eraser use and breaks visually when a background image is loaded. A true eraser would perform geometric hit-testing against existing strokes and remove intersecting entries via `splice`, at the cost of more complex CRDT operations.

Other areas that could be improved include the **fixed 800×600 canvas** with no pan or zoom, the absence of **undo/redo**, and the fact that a **Windows build** was never verified within the scope of this project.

4.4. Plans for future development

The following extensions would bring the application closer to a production-ready collaborative tool:

- **Auto-reconnect with offline queue** - detect SFU disconnection, buffer outgoing sync messages locally, and resume synchronisation automatically on reconnection.
- **Structural eraser** - implement hit-testing against the stroke list and remove intersecting strokes via CRDT `splice` instead of painting white.
- **Undo/redo** - expose Automerge's operation history to implement per-user undo that does not conflict with concurrent remote edits.
- **Resizable and scrollable canvas** - remove the fixed 800×600 constraint and allow users to pan and zoom the drawing surface.
- **Windows verification** - compile and test the release build on Windows to confirm cross-platform support as required by NF-12.

4.5. Open problems

Two problems remain unresolved and warrant further investigation. First, **document size growth**: because eraser strokes and all historical drawing operations are retained in the Automerge document, long collaborative sessions will accumulate significant data even if the visible canvas appears mostly empty. A compaction strategy - for example, periodically snapshotting the raster state and discarding operation history - would be needed for long-running use cases. Alternatively, more recent algorithms such as Eg-walker [5] address exactly this weakness of state-based CRDTs, offering substantially smaller memory footprints and faster document loading without giving up decentralised conflict resolution. Second, **client hang on SFU failure**: the current architecture has no timeout or watchdog on the LiveKit background thread, so an abrupt server crash leaves the application in an unrecoverable state that requires a manual restart. Addressing this would require either a timeout mechanism in the connection loop or a higher-level supervision strategy for the background thread.

Bibliography

[1] Automerge Contributors, *Automerge - Rust Documentation*, https://docs.rs/automerge/latest/automerge/, accessed March 2026.

[2] Conclave Team, *Conclave: A Peer-to-Peer Collaborative Text Editor*, https://conclave-team.github.io/conclave-site/, accessed March 2026.

[3] M. Kleppmann, A. Wiggins, P. van Hardenberg, M. McGranaghan, *Local-First Software: You Own Your Data, in Spite of the Cloud*, Ink & Switch, 2019, https://www.inkandswitch.com/essay/local-first/.

[4] M. Kleppmann, *Local-First Cooperation*, Onward! 2019, https://martin.kleppmann.com/2019/10/23/local-first-at-onward.html.

[5] M. Kleppmann, *Eg-Walker: Efficient Collaborative Text Editing*, 2025, https://martin.kleppmann.com/2025/04/02/eg-walker-collaborative-text.html.

[6] M. Kleppmann, A. R. Beresford, *A Conflict-Free Replicated JSON Datatype*, IEEE Transactions on Parallel and Distributed Systems, vol. 28, no. 10, pp. 2733–2746, 2017, https://arxiv.org/abs/1608.03960.

[6] LiveKit Inc., *LiveKit Self-Hosting and Deployment Guide*, https://docs.livekit.io/home/self-hosting/deployment/, accessed March 2026.

[7] LiveKit Inc., *LiveKit SFU Internals*, https://docs.livekit.io/reference/internals/livekit-sfu/, accessed March 2026.

[8] Mozilla Contributors, *WebRTC Connectivity - MDN Web Docs*, https://developer.mozilla.org/en-US/docs/Web/API/WebRTC_API/Connectivity, accessed March 2026.

[9] M. Shapiro, N. Preguiça, C. Baquero, M. Zawirski, *Conflict-free Replicated Data Types*, in: Proc. 13th Int. Symp. on Stabilization, Safety, and Security of Distributed Systems (SSS), 2011, pp. 386–400, https://hal.inria.fr/inria-00609399.

A. User Manual

Application: Collaborative Whiteboard, version 1.0

Introduction

The application enables real-time collaborative drawing on a shared virtual whiteboard. Users connect to named rooms; every participant sees changes made by others instantly.

User Interface

The main window consists of three areas:

- **Toolbar (top bar)** - drawing tools, file operations, and settings.
- **Sidebar (left panel)** - toggled by the `Menu Menu` button; contains network connection controls and chat.
- **Canvas (centre)** - the shared drawing surface.

Drawing

- **Pen / Eraser** - select the active tool from the toolbar. The eraser paints white over existing strokes.
- **Colour** - click the colour swatch in the toolbar to open the colour picker.
- **Size** - use the `Size` slider to change stroke thickness.

File Operations

- **New** - clears the canvas (prompts to save if unsaved work exists).
- **Save** - exports the current state as a `.crdt` file (full history) or `.png` (raster snapshot).
- **Open** - loads a previously saved `.crdt` file or a `.png` image as a background layer.

Collaborative Session

1. Open the sidebar with **Menu Menu**.
2. Enter a room name in the **Room** field (leave empty for a random name).
3. Click **Connect**.

4. Once connected, the button changes to **Disconnect**, and the participant list appears. Remote cursors are shown on the canvas as coloured circles labelled with the user's identity.

5. Strokes drawn by any participant appear on all other screens after the mouse button is released.

Chat

While connected, type a message in the **Message** field at the bottom of the sidebar and click **Send** to broadcast it to all participants in the room.

Troubleshooting

- **Cannot connect** - verify that the LiveKit server is running and that the `LIVEKIT_URL`, `LIVEKIT_API_KEY`, and `LIVEKIT_API_SECRET` environment variables are set correctly.
- **High latency** - on a slow connection synchronisation may take several seconds; this is expected behaviour.

B. Deployment Guide

Running the ALFA release

A pre-built binary is provided for macOS. Unzip the release archive and run the binary from a terminal with the required environment variables set (see Section B.3).

Running in a development environment

Build the application from source following the instructions in `README.md`, then start a local LiveKit server as described below. The procedure follows the official LiveKit self-hosting documentation [6].

Starting the LiveKit SFU (Docker)

```
docker run -d \
    -p 7880:7880 \
    -p 7881:7881/tcp \
    -p 7882:7882/udp \
    -e "LIVEKIT_KEYS=devkey: devsecret" \
    -e "LIVEKIT_LOG_LEVEL=info" \
    --name livekit-server \
    livekit/livekit-server \
    --node-ip $(curl -s https://checkip.amazonaws.com) \
    --bind 0.0.0.0 \
    --dev
```

Default credentials: API URL `ws://127.0.0.1:7880`, key `devkey`, secret `devsecret`.

Client configuration (`.env` file)

Create `editor/.env` with the following content:

```
LIVEKIT_API_KEY=devkey
LIVEKIT_API_SECRET=devsecret
LIVEKIT_URL=ws://127.0.0.1:7880
```

Building and running

macOS (from the `editor/` directory):

```
RUSTFLAGS="-C link-arg=-ObjC" cargo run          # debug
RUSTFLAGS="-C link-arg=-ObjC" cargo build -release  # release
```

The release binary is located at `target/release/mac_textpad`.

Windows (from the `editor/` directory):

```
cargo run            # debug
cargo build --release  # release
```

The release binary is located at `target\release\collaboratite_editor.exe`.

Distribution

Copy the release binary to the target machine together with a `.env` file (or set the three environment variables manually) and run it from a terminal. On macOS the binary can optionally be packaged into an `.app` bundle using `cargo-bundle`.

C. Manual Acceptance Test Scenarios

The following test scenarios were used for manual acceptance testing during development. Each scenario should be executed with two client instances connected to the same LiveKit room (one on the main machine, one on a second device or VM).

Table 3.1: Manual acceptance test scenarios

ID	Test step	Expected behaviour	Result
MAN-01	Launch the application	Main window opens with a blank white canvas and toolbar visible	PASS
MAN-02	Draw a freehand line with the Pen tool	Line appears immediately on the canvas with no visible delay	PASS
MAN-03	Change colour to red and size to 20	Subsequent strokes use the new colour and thickness	PASS
MAN-04	Click Save and choose a file path	System save dialog appears; file is written to disk	PASS
MAN-05	Client A enters room name `TestRoom` and clicks Connect	Status changes to Connected; room name shown in sidebar	PASS
MAN-06	Client B joins the same room `TestRoom`	Connection succeeds; Client A sees Client B in participant list	PASS
MAN-07	Client A draws a circle	Circle appears on Client B's canvas within <1 s	PASS
MAN-08	Client B moves the mouse over the canvas	Client A sees a coloured cursor labelled with Client B's identity	PASS
MAN-09	Client A sends a chat message	Message appears in Client B's event log	PASS
MAN-10	Client B clicks Disconnect	Client A receives a participant-left notification	PASS

List of appendices

1. Appendix A - User Manual
2. Appendix B - Deployment Guide
3. Appendix C - Manual Acceptance Test Scenarios